Patch Now, Pay Later? Cybersecurity Vulnerabilities and Technical Debt *

João Dimas† Labros Pnevmatikos ‡

November 9, 2026

Abstract

We study how cybersecurity vulnerabilities affect firm performance. We analyze 610 open-core software firms from 2010 to 2025, comparing firms affected by vulnerability discoveries with unaffected firms. These discoveries concern third-party dependency packages, making their timing plausibly exogenous to the firms using them. Exposed firms experience approximately 10% slower headcount growth relative to their pre-treatment average and a 1.5 percentage point decline in acquisition probability. Commit-level evidence reveals a crowding-out mechanism: the share of non-remediation development activity falls by 2.8 percentage points, and fix-related effort remains elevated for twelve months, while feature-related activity declines at longer horizons. Technical debt and cognitive complexity grow approximately 20 and 24 percentage points faster, respectively. Together, the findings suggest that cybersecurity vulnerabilities can disrupt firm growth and place persistent demands on engineering resources.

Keywords: Cybersecurity, intangible assets, technical debt, software quality, firm growth.

*We are grateful to Todd Gormley and Margarita Tsoutsoura for their invaluable comments and guidance throughout this project. We also thank Manthos Delis, Julie Zhiyu Fu, Armando Gomes, Spyros Lagaras, Asaf Manela, Emmanuel Tsiritakis, and participants of the Analysis Sync Seminar at Roblox for their insightful feedback and suggestions.
We gratefully acknowledge support from the Google Research Program.

†Olin Business School at Washington University in St Louis. Send correspondence to João Dimas; e-mail: joaodimas@wustl.edu.
‡Olin Business School at Washington University in St Louis.

1 Introduction

> *"There are two types of companies: those that have been hacked, and those who don't know they have been hacked."* — John T. Chambers, former CEO of Cisco

Software firms build their products using third-party packages that supply common functionality. A newly discovered vulnerability in one of these packages can require firms to redirect engineering resources toward urgent repairs, even when no attack has occurred. The immediate task is to remove the security risk, but updating a dependency may also require changes elsewhere in the product. These demands raise an economic question: how do cybersecurity vulnerabilities affect firm growth, and can the engineering response create costs that persist after the initial vulnerability is addressed?

Existing research establishes that cybersecurity risk matters for firms and investors. Florackis et al. (2023) show that risk measures constructed from corporate disclosures predict future cyberattacks and explain cross-sectional return variation. Such measures depend on what managers report, and firms may withhold information about severe incidents (Amir, Levi, and Livne, 2018). Crosignani, Macchiavelli, and Silva (2023) trace the economic consequences of a major cyberattack through supplier networks. These studies illuminate the pricing of cyber risk and the consequences of realized attacks. Vulnerability discoveries raise a related question: whether responding to a security flaw imposes economic costs even before it is exploited. Studying this margin requires observing both the vulnerable software and the firm's response.

A potential source of persistent costs is technical debt. It refers to the future cost of reworking code that was originally written quickly to meet short-term goals (Cunningham, 1992). As with financial debt, firms effectively "pay interest" on technical debt through elevated maintenance costs (Sculley et al., 2014). They may eventually need to refactor the code, restructuring it without changing its functionality. Technical debt is widely recognized in software engineering (Kruchten, Nord, and Ozkaya, 2012; Li, Avgeriou, and Liang, 2015), but it does not appear on financial statements. Measuring this liability requires access to the codebase and a consistent way to estimate the work needed to remedy coding problems. This makes software maintenance an important, but difficult-to-observe, margin of firm performance.

We study these questions in open-core software firms, which commercialize products whose core source code is publicly available. We identify 648 firms and obtain codebase data for 610, constructing a panel covering 2010 to 2025. The setting lets us estimate software's maintenance liabilities alongside firm growth and acquisition outcomes. We combine dependency information and vulnerability records with employment histories from LinkedIn and financing and acquisition records from PitchBook. We also analyze 84,831 monthly repository snapshots using SonarQube, an industry-standard tool for evaluating code quality. These snapshots let us measure technical debt and cognitive complexity, a measure of how difficult the code's logic is to understand and modify.

Our source of variation is the public discovery of critical vulnerabilities in third-party dependency

packages. These discoveries concern shared software components used by many firms. Their timing is therefore plausibly exogenous to the quality or performance of any one firm using the package. We identify exposure by matching firms' dependency versions to the versions affected by each vulnerability. We focus on critical-severity discoveries, rated at least nine on the Common Vulnerability Scoring System's ten-point scale. This yields 88 vulnerability events affecting 121 firms, approximately one in five firms in the broader sample.

Remediation can require developers to modify integration code, resolve incompatibilities, and test the product. Under time pressure, expedient fixes may increase future maintenance needs. Remediation may also improve existing code, and firms may absorb the extra workload by increasing total development activity rather than displacing other work. Technical debt and developers' recorded work allow us to examine how firms respond and whether the response extends beyond the initial repair.

We estimate a stacked difference-in-differences model comparing changes in firms exposed to a critical vulnerability discovery with contemporaneous changes in unaffected firms. Firm-cohort fixed effects account for persistent differences across firms within each comparison, while cohort-specific time effects absorb common changes around each discovery. The identifying assumption is that exposed and unaffected firms would have followed parallel outcome trends in the absence of the discovery. Because the discovery is external, the comparison avoids the concern that vulnerabilities in a firm's own code reflect its engineering quality.

Exposed firms experience approximately eight percentage points lower headcount growth than unaffected firms. This difference is equivalent to about 10% of exposed firms' average pre-treatment growth rate. Acquisition probability declines by 1.5 percentage points. We find no statistically significant effects on funding or survival. The findings indicate that the economic consequences of vulnerability discoveries extend beyond the immediate security task, with slower organizational growth and a lower likelihood of being acquired over the period we study.

We then examine whether these changes accompany a growing maintenance burden. Technical debt accumulates approximately 20 percentage points faster in exposed firms in the baseline specification. Cognitive complexity grows 24 percentage points faster in the baseline specification (21–24 across specifications). Together, these measures show that the affected codebases accumulate larger maintenance liabilities and become harder to work with. This pattern is consistent with time-sensitive remediation creating persistent engineering costs.

The development histories reveal both an immediate response and a longer shift in the composition of work. Individual code changes, or "commits," increase by approximately 2.7 per day in exposed repositories relative to unaffected repositories immediately following discovery. The increase attenuates in wider event windows. Commit messages containing vulnerability-resolution language, such as "upgrade," "CVE" vulnerability identifiers, and "security-patch," also become more common during the first two weeks. The concentration of this activity around discovery is consistent with an urgent remediation response.

The response continues beyond that initial burst. Across all repositories with commit histories, the share of commits containing fix-related language rises by 3.2 percentage points and remains elevated over the following twelve months. The share of non-remediation work falls by 2.8 percentage points. Feature-related commit shares decline at longer horizons in the event-study estimates, while the average change over the twelve-month window is not statistically significant ($p = 0.39$). These patterns indicate that remediation places persistent demands on engineering resources.

We also examine how cognitive complexity relates to subsequent firm performance in the broader panel. With firm, month, and programming-language fixed effects, a 10% increase in cognitive complexity is associated with approximately 1.8 percentage points lower subsequent headcount growth. This comparison connects the measured maintenance burden to firm performance.

We assess the sensitivity of the main comparisons using permutation tests, alternative clustering structures, different event windows, and leave-one-cohort-out analyses. We also include industry-segment by calendar-month fixed effects to account for changes specific to firms' product markets. Section 3.4 presents these checks alongside event-study evidence on the timing and stability of the estimated differences.

Our results contribute to two strands of the literature. First, we extend research on the economic consequences of cybersecurity risk from realized attacks and disclosure-based measures to vulnerability discoveries in shared software dependencies (Florackis et al., 2023; Amir, Levi, and Livne, 2018; Crosignani, Macchiavelli, and Silva, 2023). Direct codebase observation allows us to identify exposed firms and study their engineering response. This setting brings the operational costs of responding to vulnerabilities into the analysis of cybersecurity and firm performance.

Second, we contribute to research on intangible assets and the liabilities associated with software. Prior work documents the importance of intangible capital for firm value and growth (Ewens, Peters, and Wang, 2025) and the pricing of organization capital (Eisfeldt and Papanikolaou, 2013). Software assets also require future maintenance, a liability that is difficult to observe in conventional financial data. Linking a dollar-denominated measure of technical debt to vulnerability discoveries shows how an external vulnerability discovery changes the future cost of maintaining a firm's software, with applications to valuation and M&A due diligence.

The remainder of the paper proceeds as follows. Section 2 describes our data sources, sample construction, and measurement approach for both vulnerability discoveries and technical debt. Section 3 presents our empirical framework and main results, including the firm outcomes, immediate remediation response, crowding-out evidence, code-quality consequences, and robustness tests. Section 4 concludes.

2 Data & Background

The primary challenge in studying firms' technological assets is that codebases are typically proprietary and accessible only to internal stakeholders. To overcome this, we focus on firms that utilize an open-core business model—companies that make their core product code publicly available while monetizing through enterprise features and support services. This approach allows us to link complete codebase histories to firm-level outcomes over a 15-year period.

2.1 Sample Construction and Firm Characteristics

Open-source software (OSS) has become foundational infrastructure for the global economy. Industry audits reveal that 96% of commercial codebases contain open-source components, and that open source constitutes approximately 77% of all code in the average application (Synopsys, 2024). Hoffmann, Nagle, and Zhou (2024) estimate the demand-side value of widely used OSS at $8.8 trillion—implying that firms would need to spend 3.5 times more on software development if these components did not exist. This pervasiveness means that software produced by even small open-source firms can become deeply embedded in the technology stacks of large enterprises, financial institutions, and government agencies. A vulnerability in a widely adopted open-source database engine or authentication library does not merely affect its developer; it propagates downstream to every organization that depends on the product, creating systemic exposure across the software supply chain (Crosignani, Macchiavelli, and Silva, 2023).

The firms in our sample are precisely the producers of such foundational software. They operate under an open-core business model, which combines the collaborative benefits of OSS with revenue opportunities from commercial licensing and support. Under this model, companies release their fully functional core product under an open-source license—making the code freely accessible to developers, customers, and researchers—while maintaining exclusive rights to license enterprise-grade features, modules, or dedicated support services through paid subscriptions. The organizational literature highlights OSS as a central driver of innovation in the modern economy (Conti, Peukert, and Roche, 2025; Conti et al., 2023; Fershtman and Gandal, 2011), and many companies transition from fully open-source projects to the open-core model as they seek to generate revenue while preserving community engagement and collaborative development (Nagle, 2019). A tangible example is *RStudio* (now *Posit*), the primary development environment for the *R* programming language used by researchers, data scientists, and analysts across industries. The software is freely available and open source, but the company monetizes enterprise deployment through commercial licenses and a cloud platform. Crucially, a security flaw in RStudio's codebase would affect not only Posit as a firm but also the thousands of organizations whose analytical workflows depend on the product.

Although releasing product code publicly may appear to invite competition, this strategy offers

significant advantages: public availability accelerates adoption, fosters external contributions from developers worldwide, and builds trust through transparency. When individuals learn to use freely available software, they often advocate for enterprise adoption at their employers, creating a pathway to commercial sales. Crucially for our research, this business model grants us access to detailed codebase histories that would otherwise remain confidential within traditional software companies.

Our sample universe is the set of companies that commercialize OSS repositories identified in Dimas and Pnevmatikos (2026) through OSSRank, a leading platform in the OSS ecosystem. This process yields 648 unique companies and the 734 distinct open-source projects they develop and commercialize. As Appendix Table 18 illustrates, these firms produce foundational technology products (databases, cloud infrastructure, security tools, developer platforms, and analytics systems) that serve as building blocks in the software supply chains of larger organizations. Our firms are primarily based in the United States (62%; Appendix Table 19), with meaningful representation from European and Asian technology hubs, and operate predominantly in the technology sector. They are typically small, private, and frequently venture capital–backed. The median firm has 33 employees in its latest observed month and was founded in 2018, with founding years ranging from 2009 (10th percentile) to 2021 (90th percentile). Traditional financial disclosures are not publicly available for these private firms.

While these firms are small relative to publicly traded technology companies, two features make them well suited for studying the economic consequences of cybersecurity vulnerabilities. First, their products are widely adopted downstream: open-core databases, monitoring tools, and security libraries are integrated into the technology stacks of Fortune 500 companies, financial institutions, and critical infrastructure operators. A vulnerability in one of these products creates exposure not only for the producing firm but for every downstream adopter—precisely the supply-chain propagation channel that makes cybersecurity economically consequential (Hoffmann, Nagle, and Zhou, 2024). Second, small firms are disproportionately vulnerable to the operational disruption that vulnerability remediation entails, because they lack the redundant engineering capacity that larger organizations can deploy during crises. The mechanism we study—cybersecurity vulnerabilities diverting engineering resources—applies to any organization maintaining a software codebase, regardless of size. Moreover, firms of this size are typical of the software industry: in 2022, 97% of U.S. software publishers had fewer than 500 employees, and 81% had fewer than 20 (U.S. Census Bureau, 2025).

Appendix Table 18 presents the distribution of firms across product categories. We employ two large language models (Gemini and GPT) to classify each firm's core offering into one of ten product categories. The models agree in 86% of cases classified by both.[1] The sample exhibits substantial diversity: approximately 24% of firms develop developer tools and platforms, 15% provide cloud and infrastructure services, 14% focus on databases and storage, and 14% specialize in analytics and

[1] We retain the common classification when the models agree, use Gemini's classification when they disagree, and use the available classification when only one model classifies the firm.

observability. The remaining firms span security, enterprise applications, artificial intelligence and machine learning, collaboration, and testing.

2.2 Codebase Data

We match each project identified through OSSRank to its repository by searching GitHub and GitLab, the industry-standard platforms for software version control. These platforms maintain complete version histories through a system of *commits*. A commit is a discrete submission of code changes to the project—analogous to saving a new version of a document, but with comprehensive metadata attached. Each commit records exactly which lines of code were added, modified, or deleted; a timestamp; the identity of the contributor; and a *commit message*—a brief text description written by the developer explaining the purpose of the change (e.g., "Fix login bug" or "Add user authentication"). This granular audit trail allows us to observe not only the state of each codebase at any point in time, but also the intensity and content of development activity.

We build a monthly panel by extracting the final observed commit from each active repository-month from 2010 to 2025, following the panel construction in Dimas and Pnevmatikos (2026). Most repositories are hosted on GitHub, with a minority on GitLab; we combine both sources given the platforms' functionally equivalent version control features. We observe commit-level development activity for 693 of the 720 matched repositories, corresponding to 610 distinct firms. Code quality and technical debt measures from SonarQube static analysis are available for 545 repositories of 488 firms whose programming languages and build configurations support the analysis (Section 2.6). Firm-level analyses requiring technical debt, cognitive complexity, and other code quality measures draw from these 488 firms. We sum lines of code and cognitive complexity across each firm's repositories to construct these firm-month measures.

2.3 Employee and Funding Data

To track firm outcomes, we construct monthly headcount series from LinkedIn employee-position histories, counting employees associated with each company identifier in each month. We also classify software engineering roles from job titles to construct technical headcount. The employee panel spans January 2010 through October 2022; code histories extend through 2025. Firm-months with missing or nonpositive headcount do not enter the logged employment specification.

We supplement employee data with funding and acquisition information from PitchBook, a leading private capital markets database. We link our sample firms with data on funding rounds (Seed Round, Early Stage VC (Series A, B, C), Late Stage VC (Series D+), PE Growth/Expansion, Accelerator/Incubator, Angel (individual)), initial public offerings (IPOs), mergers and acquisitions, and business status indicators (alive/not-alive). For each event, we observe the date and transaction type, enabling us to construct indicators for whether a firm has received funding, been acquired, or

ceased operations. These outcome measures serve as proxies for firm performance in the absence of traditional financial statements, which are not publicly available for the private firms in our sample.

2.4 Vulnerability Data

For our causal analysis, we identify cybersecurity shocks that are exogenous to firm quality by focusing on vulnerabilities affecting third-party software packages that firms integrate into their code. These dependency vulnerabilities directly translate to security flaws in firms' core products, but crucially, their discovery is plausibly unrelated to any individual firm's engineering practices or management quality.

We draw vulnerability data from two complementary industry-standard sources. The GitHub Vulnerability Advisory Database, maintained by GitHub, serves as the primary vulnerability reporting mechanism for the open-source software ecosystem. When security researchers or maintainers discover vulnerabilities, they report them through this centralized system, creating a standardized, time-stamped record of public discovery. The National Vulnerability Database (NVD), operated by the U.S. National Institute of Standards and Technology (NIST), provides the official government repository for vulnerability information and supplies the authoritative scoring and classification used by enterprises and security vendors worldwide. While maintained by a U.S. government agency, the NVD catalogs vulnerabilities in software packages used globally, making it the international standard for vulnerability tracking regardless of where affected firms are located. Both databases score vulnerabilities using the Common Vulnerability Scoring System (CVSS), an industry-standard framework that quantifies vulnerability severity on a 0–10 scale.

We source our data from the official GitHub Advisory Database repository, which incorporates merged vulnerability information from the NVD. To ensure accuracy and avoid false positives, we specifically utilize the "GitHub reviewed" subset of this database. Focusing solely on critical dependency vulnerabilities affecting the software our firms use, we identify 88 unique critical cybersecurity events that impacted 121 firms via their dependencies. Appendix Tables 14–16 summarize the broader GitHub-reviewed advisory universe from which we draw our shocks, including the distribution across severity levels, software ecosystems, and years.

The CVSS framework provides a standardized way to measure vulnerability severity based on characteristics such as how easily a vulnerability can be exploited (e.g., remotely over a network versus requiring physical access) and the potential impact on system confidentiality, integrity, and availability. Scores range from 0 to 10, with vulnerabilities scoring 9.0 or above classified as "critical." CVSS Base scores reflect the intrinsic severity of a vulnerability under standardized assumptions. By restricting our sample to critical-severity vulnerabilities, we study high-severity discovery events that are comparable in potential impact and likely to prompt organizational response. Appendix A.1 explains the metrics, scoring, and use of different CVSS versions.

2.5 Matching Shocks to Firms

To identify which firms are affected by a vulnerability, we map each firm's software dependencies and their versions over time. Firms may retain an older package version because functionality they need is absent from a later release. Exact versions matter: a vulnerability might affect versions 1.2.0 through 1.4.5 but not 1.5.0 or later. We use Syft, an industry-standard software composition analysis tool, to scan each codebase and generate a complete *Software Bill of Materials* (SBOM)—a catalog of its components and their versions.

We then match each critical-severity vulnerability shock to the firm that commercializes the affected repository.

2.6 Technical Debt: Background & Measurement

The development of new software products or features frequently generates technical debt (TD). The software engineering literature defines technical debt as coding decisions that accelerate short-run development at the expense of future changes and higher maintenance costs (Sculley et al., 2014). Like financial debt, firms "pay interest" through elevated future maintenance expenses and eventually face costly "principal paydowns" when refactoring becomes unavoidable. Recent estimates place the aggregate stock of U.S. technical debt at approximately $1.5 trillion, underscoring its economic significance as an intangible liability (Krasner, 2022).

A central challenge for financial economists is that technical debt remains largely unobservable to external stakeholders. Two firms may deliver functionally identical products from a user's perspective, yet differ dramatically in their accumulated software liabilities. Because direct measurement requires continuous access to proprietary source code, only internal developers and managers can accurately assess TD magnitude. This opacity makes technical debt difficult to contract upon, monitor, or price in financial markets, despite its economic importance. Survey evidence illustrates this burden: developers allocate approximately 13.5 hours per week to managing technical debt (Stripe, 2018).

To measure TD, we use SonarQube to identify maintainability violations and estimate the time needed to repair them (Letouzey, 2012; Dimas and Pnevmatikos, 2026). Each violation receives a model-implied remediation time: a small documentation fix may take minutes, while a structural problem may require hours. Summing these times across the codebase produces its estimated repair time, reported in minutes. We apply the same procedure to each monthly code snapshot to track changes in the maintenance burden.

We express this remediation burden in dollars by applying the calibration proposed in Dimas and Pnevmatikos (2026) to the estimated repair time in minutes. The calibration uses a benchmark of 20% of software replacement cost (McKinsey & Company, 2023), using 30 minutes of development time per line and an hourly engineering wage of $55. The dollar stock uses a common multiplier of approximately $33.04 per minute of estimated repair time. Because this factor is common across firms

and time, it changes the units while preserving comparisons of proportional changes. Our regressions use ln(Technical Debt) on strictly positive stocks. Appendix A.3 gives the formula.

Beyond technical debt, we employ *Cognitive Complexity* as a complementary measure of maintainability. While technical debt captures the estimated remediation cost of code issues, cognitive complexity quantifies the mental effort required to understand and modify code. This metric was developed by SonarSource and is implemented in SonarQube (Campbell, 2023). In our regression analysis, we winsorize cognitive complexity at the 1st and 99th percentiles to limit the influence of outliers. Appendix A.4 provides a detailed explanation and discussion of this metric.

2.7 Summary Statistics

Table 1 summarizes firm size, codebase scale, and maintenance measures in the firm-month panel.

Table 1: Summary Statistics: Firm-Month Panel

		Distribution						
Variable	**N**	**Mean**	**SD**	**Min**	**P25**	**Median**	**P75**	**Max**
Panel A: Outcome Variables								
Ln(Headcount)	27,827	3.70	2.64	0.00	1.87	3.22	5.03	12.67
Ln(Software Engineer Headcount)	23,577	3.16	2.38	0.00	1.61	2.71	4.28	11.22
Funding Round (0/1)	53,593	0.01	0.11	0.00	0.00	0.00	0.00	1.00
Acquisition (0/1)	53,593	0.04	0.20	0.00	0.00	0.00	0.00	1.00
Is Alive (0/1)	53,593	0.98	0.13	0.00	1.00	1.00	1.00	1.00
Panel B: Technical Debt								
Ln(Technical Debt in $)	53,593	12.16	2.68	4.19	10.64	12.50	14.08	19.03
Technical Debt per Line of Code	53,593	8.83	13.76	0.00	3.24	5.92	10.22	660.87
Technical Debt Velocity	45,140	5.68	45.85	0.00	1.14	3.03	5.36	5,176.80
Panel C: Code Quality								
Ln(Lines of Code)	53,593	10.93	2.42	0.00	9.66	11.19	12.53	16.84
Ln(Open Issues)	53,593	6.83	2.51	0.00	5.35	7.08	8.63	13.75
Ln(Cognitive Complexity)	52,170	8.67	2.62	0.00	7.29	8.99	10.44	15.21
Ln(New Security Issues - High)	3,588	0.83	0.80	0.00	0.00	0.69	1.10	5.06
New High-Severity Security Issues	53,593	0.25	2.35	0.00	0.00	0.00	0.00	158.00
Panel D: Firm Characteristics								
Founding Year	37,884	2,015.1	7.6	1,951	2,013	2,017	2,020	2,024

Note: This table presents summary statistics for the firm-month panel. The sample includes up to 53,593 firm-month observations; headcount variables have fewer observations due to missing LinkedIn data. Logged variables use strictly positive values, so observation counts vary by variable. Detailed variable definitions are provided in the appendix.

3 Empirical Framework and Results

3.1 Identification and Estimation

A central challenge in linking cybersecurity vulnerabilities to firm outcomes is omitted variable bias. For instance, firms with better management may invest more in security and engineering practices and also grow faster for reasons unrelated to cybersecurity. As a result, simple correlations between vulnerability exposure and firm performance need not reflect causal effects.

We address this concern by exploiting public discoveries of vulnerabilities in third-party software dependencies used by firms in our sample. Once a vulnerability is discovered, any firm that happens to rely on an affected package version is exposed regardless of its own engineering quality or business strategy. Using our monthly codebase snapshots, we match the dependency versions recorded at the final commit of the disclosure month to the affected-version ranges in the advisory data. Exposure therefore reflects each repository's dependencies at the end of the disclosure month. We date each event by its NVD publication date where available and otherwise by its advisory publication date.

We focus on discoveries of critical-severity vulnerabilities, defined as vulnerabilities with CVSS scores of 9 or above. Appendix Table 12 defines the CVSS measure and our critical-severity threshold. In our sample, we identify 88 unique critical-severity vulnerability events affecting 121 firms through their third-party dependencies. These shocks are not infrequent: over our sample period, approximately one in five firms experiences at least one critical dependency vulnerability. Each event, defined by a dependency and its discovery month, can form a cohort. Because we date each firm's treatment at its first critical event and apply the sample restrictions described below, 38 of the 88 events form the cohorts of our stacked design. Appendix Figure 14 shows that the temporal distribution of these cohort events broadly mirrors the rising trend in critical vulnerability discoveries affecting our firms' dependencies, suggesting that our sample of shocks is representative of these disclosures.

Each shock requires the affected firm to either upgrade to a patched version of the dependency (which may introduce compatibility issues with existing code), downgrade to a pre-vulnerability version (potentially losing functionality), or refactor the codebase to remove the dependency entirely. The scope of this remediation effort depends on how deeply the vulnerable package is integrated into the firm's product, but even straightforward dependency upgrades frequently trigger cascading changes across the codebase. These events are economically salient and difficult to ignore: even absent a realized breach, firms must devote engineering resources to remediation.

We estimate treatment effects using a stacked difference-in-differences (DiD) framework. We adopt this approach to address the staggered timing of vulnerability disclosures, where treatment occurs at different times across firms. A large body of econometric research demonstrates that, in the presence of dynamic treatment effects, standard two-way fixed effects (TWFE) estimators can be biased because they implicitly use already-treated units as controls for later-treated units, which can yield misleading inferences and, in some cases, produce estimates with the wrong sign (Callaway and Sant'Anna, 2021; Sun and Abraham, 2021; Baker, Larcker, and Wang, 2022). Although this issue has been emphasized more recently in the econometrics literature, the underlying concern and cohort-based solutions were already recognized in earlier applied work (Gormley and Matsa, 2011, 2016). Our stacked design circumvents this issue by constructing a "clean" counterfactual, ensuring that treated firms are compared strictly to not-yet-treated controls within the same calendar window (Gormley and Matsa, 2011). Identification requires two assumptions: (1) parallel trends—in the absence of treatment, treated and control firms would have followed parallel outcome paths; and (2)

no anticipation—firms do not adjust behavior before the public discovery of the vulnerability. The second assumption is supported by the nature of our shocks: vulnerability discoveries are made by independent security researchers at unpredictable times, and firms have no advance knowledge of when a vulnerability in their dependency will be publicly disclosed.

To implement the stacked design, we match each critical-severity discovery to affected firms using the dependency–version mapping described in Section 2.5. We retain discoveries classified as requiring a major or minor dependency upgrade. Starting from our universe of 648 firms, we apply four sample restrictions. First, we restrict to firms that commercialize a single repository (their core product), ensuring that treatment status is unambiguous; this excludes firms whose exposure might be diluted across multiple products. Second, we require that firms have non-missing headcount data from LinkedIn during the event window, which drops firms for which we cannot observe the outcome variable. Third, a control firm's first qualifying critical dependency vulnerability must be discovered at least 12 months after the cohort event, or the firm must have no such discovery. Fourth, we require that firms have valid SonarQube static analysis results with positive lines of code, which is necessary for the technical debt and code quality outcomes that form the core of our mechanism analysis. Treated firms are those whose codebase contained the affected dependency version at the time of discovery. The resulting stacked dataset contains 284 firms (117 treated; 167 control) across 38 cohorts and 114,393 firm-cohort-month observations. Table 2 reports summary statistics for the stacked dataset.

Table 2: Summary Statistics: Stacked Difference-in-Differences Sample

		Distribution						
Variable	**N**	**Mean**	**SD**	**Min**	**P25**	**Median**	**P75**	**Max**
Panel A: Outcome Variables								
Ln(Headcount)	114,393	3.33	2.04	0.00	1.95	3.09	4.56	11.93
Funding Round (0/1)	114,393	0.02	0.15	0.00	0.00	0.00	0.00	1.00
Acquisition (0/1)	114,393	0.05	0.22	0.00	0.00	0.00	0.00	1.00
Is Alive (0/1)	114,393	0.98	0.13	0.00	1.00	1.00	1.00	1.00
Panel B: Technical Debt								
Ln(Technical Debt)	114,393	12.03	2.68	4.60	10.08	12.10	14.01	18.54
Panel C: Code Quality								
Ln(Lines of Code)	114,393	10.40	2.39	1.10	9.02	10.51	12.12	14.93
Ln(Open Issues)	114,393	6.65	2.55	0.00	4.85	6.78	8.52	13.29

Note: This table presents summary statistics for the stacked difference-in-differences sample. The sample includes 284 firms (117 treated, 167 control) across 38 cohorts. Detailed variable definitions are provided in the appendix.

Before turning to estimation, we assess the comparability of firms treated in at least one cohort and firms never treated. Table 3 reports pre-treatment means of key observables for these two groups at $t = -1$ in the stacked DiD sample. Technical debt, security hotspots, open issues, high-severity security issues per thousand lines of code, bugs per thousand lines of code, duplicated lines, and funding rates are all well-balanced across groups, with normalized differences well below the 0.25 threshold recommended by Imbens and Rubin (2015). Ever-treated firms display somewhat lower

pre-treatment cognitive complexity (normalized difference of −0.39) and fewer critical violations (−0.21), indicating less complex, more maintainable codebases. All level differences are absorbed by firm-by-cohort fixed effects. Ever-treated firms are somewhat smaller as measured by headcount (normalized difference of −0.42); we include lines of code as a size control. The full sample attrition funnel is reported in Appendix Table 27.

Table 3: Pre-Treatment Balance: Ever-Treated vs. Never-Treated Firms

	Ever-treated firms	**Never-treated firms**	**Diff.**	***p*-value**	**Norm. Diff.**
Ln(Technical Debt)	12.01	11.99	0.02	0.86	0.01
Ln(Cognitive Complexity)	7.97	8.96	-0.99	<0.01	**-0.39**
Ln(Headcount)	2.63	3.42	-0.79	<0.01	**-0.42**
Ln(Lines of Code)	10.26	10.42	-0.17	0.07	-0.07
Ln(Security Hotspots)	3.76	4.00	-0.25	<0.01	-0.11
Ln(Open Issues)	6.68	6.61	0.07	0.48	0.03
Ln(Critical Violations)	5.52	6.07	-0.55	<0.01	-0.21
Vulnerabilities / kLoC	0.10	0.56	-0.46	<0.01	-0.20
Security Issues, High / kLoC	0.02	0.02	0.01	0.23	0.06
Bugs / kLoC	3.23	2.68	0.55	0.15	0.06
Duplicated Lines	47,347	46,693	654	0.93	0.00
Funding Round	0.02	0.02	0.00	0.87	-0.01
N (firm×cohort obs.)	696	4377			

Note: This table reports pre-treatment means at $t = -1$ (one month before vulnerability discovery) for firms treated in at least one cohort and firms never treated in the stacked DiD panel. Observations are firm-by-cohort pairs. Logged variables use strictly positive observations; their row-specific samples can be smaller than the full group counts shown. The normalized difference follows Imbens and Rubin (2015): $(\bar{X}_T - \bar{X}_C)/\sqrt{(s_T^2 + s_C^2)/2}$, where $\bar{X}_T$ and $\bar{X}_C$ are the sample means, and s_T^2 and s_C^2 are the sample variances for the ever-treated and never-treated groups, respectively. Values below 0.25 in absolute value suggest adequate balance; bold entries exceed this threshold. The p-value is from Welch's t-test allowing for unequal variances. Security issues (High) use SonarQube's alternative (Multi-Quality Rule) classification and are not a subset of the vulnerabilities row, which uses the traditional classification.

We estimate the following stacked difference-in-differences specification:

$$\ln(\text{Headcount})_{i,c,t} = \beta \cdot did_{i,c,t} + \gamma' X_{i,c,t} + \delta_{c,t} + \chi_{c,i} + \varepsilon_{i,c,t} \tag{1}$$

The dependent variable, $\ln(\text{Headcount})_{i,c,t}$, is the log headcount of firm i in cohort c at event time t. The treatment indicator $did_{i,c,t}$ equals $\text{Treated}_{i,c} \times \text{Post}_{c,t}$, where $\text{Post}_{c,t}$ indicates periods strictly after the disclosure month within cohort c (i.e., $t \geq 1$; the discovery month itself is coded as pre-period since exposure is partial during that month). We include time-varying firm and project controls $X_{i,c,t}$, firm-by-cohort fixed effects $\chi_{c,i}$, and event-time-by-cohort fixed effects $\delta_{c,t}$, so identifying variation comes from within-cohort changes in treated firms relative to not-yet-treated controls at the same event time. In the baseline specification, the coefficient aggregates the within-cohort DiD contrasts with weights proportional to the sum of squared treatment residuals after removing the fixed effects.

These weights reflect each cohort's treated–control composition and observed pre- and post-disclosure coverage. We report the observation support and implied cohort weights in the replication package.

Standard errors are clustered at the firm level, which accounts for serial correlation within firm and for the fact that control firms may appear in multiple cohorts, inducing mechanical correlation across cohorts. Appendix Table 25 shows that results are robust to alternative clustering structures, including two-way clustering at the firm and cohort level.

3.2 Firm Outcomes

3.2.1 Headcount

The sign of the headcount effect is not obvious *a priori*: firms could hire to handle remediation, or remediation could divert resources from growth-oriented activities and slow hiring. The net effect is an empirical question.

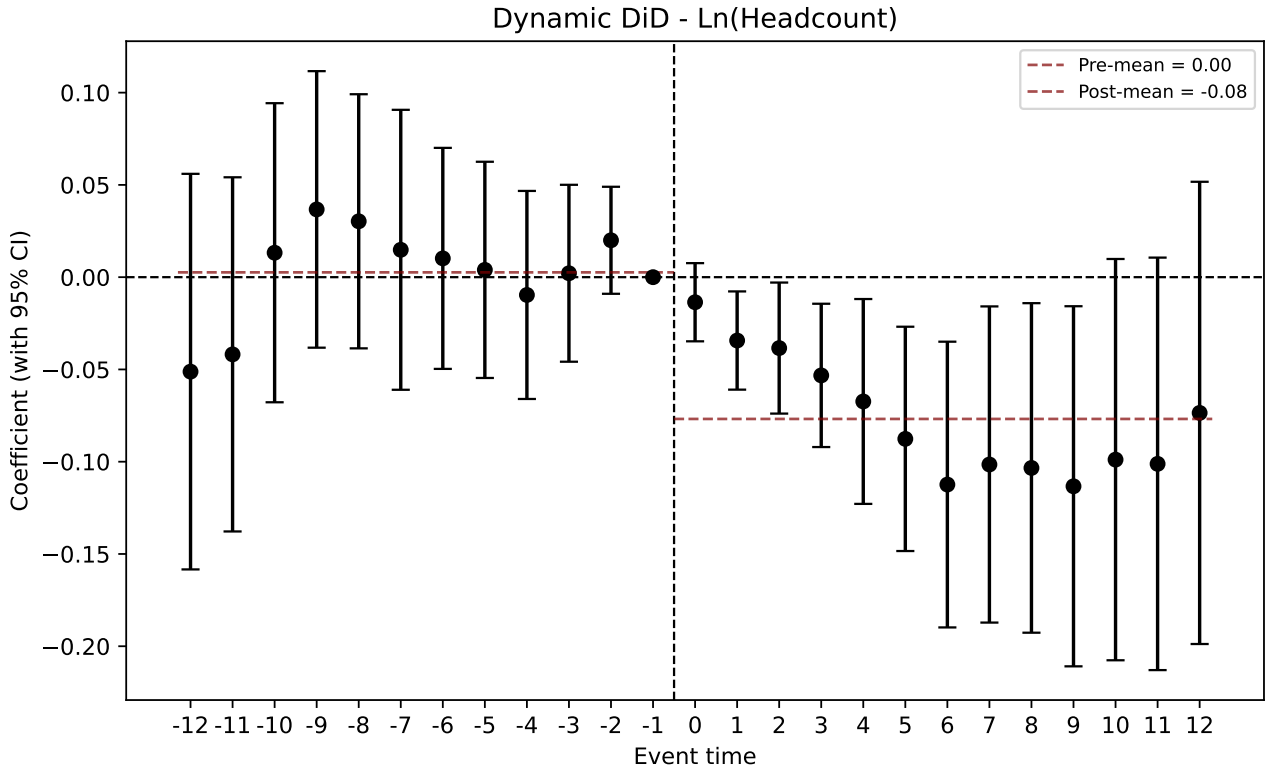


Figure 1: Cybersecurity Vulnerabilities and Firm Growth (Event Time in Months). Points show monthly stacked DiD estimates for log headcount in the firm-level estimation sample, relative to month −1 before critical vulnerability discovery. Coefficients are in log points. The specification includes firm-by-cohort and event-month-by-cohort fixed effects. Error bars show 95% confidence intervals using firm-clustered standard errors. Maroon horizontal segments show average coefficients before and in or after the discovery month.

Table 4: Difference-in-Differences Estimates for Headcount

Dep Var: **Ln(Headcount)**	(1)	(2)	(3)	(4)
Treated × Post	-0.08*	-0.08*	-0.08*	-0.09**
	(0.04)	(0.04)	(0.04)	(0.04)
# Lines of Code		0.00	0.00	0.00
		(0.00)	(0.00)	(0.00)
Firm × Cohort FE	Yes	Yes	Yes	Yes
Relative Month × Cohort FE	Yes	Yes	Yes	Yes
Language × Cohort FE			Yes	
Firm Type × Month FE				Yes
R^2	0.98	0.98	0.98	0.98
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on firm headcount over a ±12 month event window. The dependent variable is the natural log of firm headcount (from LinkedIn). The sample consists of 284 firms (117 treated, 167 control) across 38 cohorts. Treated firms are those whose codebase contained an affected dependency version at the time of a critical-severity disclosure (CVSS ≥ 9). Controls follow the eligibility rules described in Section 3. Column (1) is the baseline specification with firm-by-cohort and event-time-by-cohort fixed effects only. Column (2) adds lines of code as a time-varying control for project scale. Column (3) further absorbs programming-language-by-cohort fixed effects. Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects, where firm types group the ten product categories classified by large language models in Table 18 into five types (Infrastructure, Developer Tools, Security, Data & Analytics, Business Applications). Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

Figure 1 shows the monthly dynamic treatment effects on headcount growth around the discovery month.[2] One concern is that the slightly positive pre-discovery coefficients and the subsequent decline could reflect mean-reversion in headcount growth among smaller firms (treated firms are somewhat smaller; see Table 3). However, this interpretation is inconsistent with the pattern in Figure 1, where the pre-discovery coefficients are approximately flat rather than declining, and the decline begins in the discovery month ($t = 0$)—consistent with a discrete shock rather than a reversion already under way. Leave-one-cohort-out analysis confirms that the result is stable across all 38 cohorts (coefficient range: -0.07 to -0.09; see Section 3.4). After discovery, treated firms experience a persistent decline in growth velocity relative to controls.

Table 4 reports the corresponding regression estimates. Column (1) is the baseline specification with only firm-by-cohort and event-time-by-cohort fixed effects (no additional time-varying controls) and yields a coefficient of -0.08 ($p = 0.05$). Column (2) adds controls for project scale measured by lines of code; the estimate remains stable at -0.08 ($p = 0.05$). Column (3) further absorbs programming-language-by-cohort fixed effects to address compositional differences across treatment arms within each cohort; the estimate is virtually unchanged at -0.08 ($p = 0.05$).[3] Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects, absorbing industry-specific time trends; the estimate is -0.09 ($p = 0.04$).

In the baseline specification (column 1), treated firms' headcount grows 8.1 percentage points slower than controls, implying 7.8% lower headcount levels relative to the counterfactual.

To benchmark this magnitude, treated firms' headcount grew by an average of 79.7% over the twelve months before discovery; the 8.1-percentage-point slowdown equals about 10% of that growth rate. For the median firm-month among the 284 sample firms (19 employees), the 7.8% lower headcount level corresponds to approximately 1.5 fewer employees—a meaningful reduction for small teams operating with limited engineering capacity.

3.2.2 Acquisition

We also examine acquisition probability. The outcome records whether a firm has been acquired (merger/acquisition or buyout/LBO), taking the maximum of this indicator separately within each pre- and post-discovery phase. Table 5 shows declines of 1.5–1.7 percentage points across specifications (largest $p = 0.01$).

[2] The pre-discovery coefficients are economically small, though a formal joint test of exact parallel pre-trends on the eleven monthly pre-treatment coefficients gives $W(11) = 17.4$ ($p = 0.09$).

[3] For the headcount specification, we use language-by-cohort fixed effects in Column (3) rather than language-by-month, because headcount is available only for a subset of firms and the language-by-month cells can become sparse. The acquisition, funding, technical-debt, and cognitive-complexity tables use language-by-month fixed effects in Column (3).

Table 5: Difference-in-Differences Estimates for Acquisition

Dep Var: **Acquisition (Pre/Post)**	(1)	(2)	(3)	(4)
Treated × Post	-1.52***	-1.75**	-1.75**	-1.61**
	(0.58)	(0.74)	(0.74)	(0.63)
# Lines of Code			0.00	0.00
			(0.00)	(0.00)
Firm × Cohort FE	Yes	Yes	Yes	Yes
Relative Month × Cohort FE	Yes	Yes	Yes	Yes
Language × Month FE		Yes	Yes	
Firm Type × Month FE				Yes
R^2	0.95	0.95	0.95	0.95
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on acquisition probability over a ±12 month event window. The dependent variable records whether the firm has been acquired, taking the maximum separately within each pre/post phase. Coefficients and standard errors are in percentage points. The sample and treatment assignment follow Table 4. Column (1) includes firm-by-cohort and event-time-by-cohort fixed effects only. Column (2) adds language-by-month fixed effects. Column (3) further controls for project scale (lines of code). Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

Relative to the unconditional acquisition rate of 5.8%, the baseline estimate of 1.5 percentage points represents a 26.0% relative decline in acquisition probability (29.9% using the 1.7-percentage-point estimate from column 3), a meaningful reduction in exit opportunities for firms in this segment. One interpretation is that critical vulnerabilities reduce firms' attractiveness as acquisition targets by increasing perceived technical risk and the expected integration burden, particularly if remediation generates technical debt or slows product progress. The estimate is stable under firm-type-by-calendar-month fixed effects (column 4: 1.6 percentage points, $p = 0.01$).

3.2.3 Funding and Survival

We also examine funding and survival. Point estimates on both outcomes are small and not statistically significant across all specifications (Appendix Tables 21 and 22; Appendix Figure 13). This pattern is consistent with shocks that stunt growth rather than threaten firm continuation.

Taken together, the firm-level evidence reveals that critical vulnerability shocks impose real organizational costs: treated firms experience slower headcount growth and are less likely to exit via

acquisition. A natural question is *why* these performance effects arise. Our codebase-level data allow us to examine the engineering response.

3.3 The Remediation Channel

We organize the evidence in three parts: the immediate emergency response, the resulting crowding-out of innovation-oriented development, and the consequences for code quality.

3.3.1 Immediate Remediation

We begin by providing direct evidence that vulnerability discoveries trigger time-sensitive remediation efforts. Our argument is that these shocks generate elevated development activity that, due to time pressure, diverts engineering resources from productive work. We analyze repository-level commit activity around vulnerability discoveries.

We first examine whether vulnerability discoveries trigger an immediate increase in development activity. Using daily-frequency specifications over narrow event windows, we estimate the treatment effect on raw commit counts. Figure 2 presents dynamic treatment effects for total commits in a ±3 day window around vulnerability discovery. Unlike the firm-level analysis, Post includes the discovery day itself, since remediation can begin the same day. The coefficient on Treated × Post is positive, indicating that treated repositories in the firm-level DiD sample produce approximately 2.7 more commits per day than control repositories in the days immediately surrounding discovery ($p = 0.01$). This effect attenuates in wider windows (Table 6), to 1.2 additional commits at ±7 days ($p = 0.06$) and 0.5 additional commits at ±14 days, which is not statistically significant ($p = 0.18$). This pattern is consistent with the increase in development activity being concentrated in the immediate aftermath of discovery.

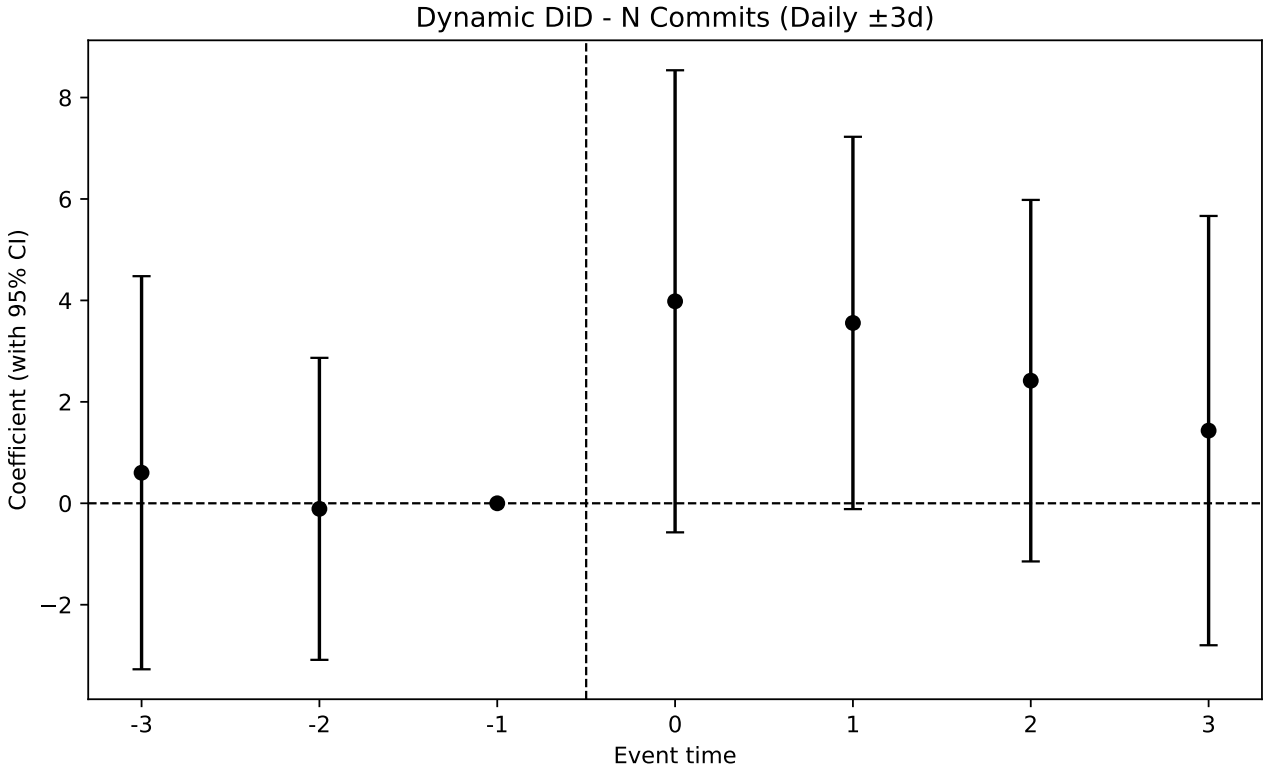


Figure 2: Immediate Commit Response to Vulnerability Discovery (±3 Days). Points show daily stacked DiD estimates for total commits per repository-day, using repositories in the firm-level stacked DiD sample. Estimates are relative to day −1 before critical vulnerability discovery and include repository-by-cohort and event-day-by-cohort fixed effects. Error bars show 95% confidence intervals using repository-clustered standard errors.

The commit intensity results in Table 6 show that treated repositories produce more commits, but they do not distinguish routine development from remediation-specific work. To isolate the remediation channel, we classify each commit as *vulnerability-resolution related* if its message contains any of the terms *bump*, *upgrade*, *CVE-**, *CVSS*, *vulnerability*, *security-fix*, or *security-patch*. These terms capture version bumps and security-specific patches—the direct mechanical response to a discovered vulnerability—while excluding general development language. Approximately 3.5% of all commits in our sample contain at least one such term.

Table 7 reports the results. In Panel A, treated repositories exhibit an increase in vulnerability-resolution language at the ±7 and ±14 day windows: 1.6 percentage points at ±7 days ($p < 0.01$) and 1.1 percentage points at ±14 days ($p = 0.01$). The ±3 day estimate of 1.4 percentage points is not statistically significant ($p = 0.16$).

Panel B shows a similar pattern: the ±3 day estimate of 1.7 percentage points is again not statistically significant ($p = 0.25$), while the ±14 day estimate is 1.5 percentage points ($p = 0.01$). The ±30 day column provides a falsification-style test: the coefficient attenuates to near zero and is not statistically significant in either panel ($p = 0.87$ and $p = 0.41$ in Panels A and B, respectively),

Table 6: Daily Difference-in-Differences Estimates for Commit Intensity

	±3 Days	±7 Days	±14 Days
Panel A: All Analyzed Repositories			
Commits (Count)	1.61**	0.47	0.25
	(0.70)	(0.36)	(0.27)
# Obs	45,858	75,288	128,700
Panel B: Stacked DiD Repositories			
Commits (Count)	2.70**	1.19*	0.51
	(1.14)	(0.63)	(0.39)
# Obs	13,787	22,823	39,359
Repo × Cohort FE	Yes	Yes	Yes
Relative Day × Cohort FE	Yes	Yes	Yes

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on daily commit intensity. The dependent variable is the raw number of commits per repository-day. Panel A uses all repositories with commit histories; Panel B restricts the sample to repositories included in the firm-level stacked DiD analysis. Standard errors in parentheses are clustered at the repository level. * p<0.1, ** p<0.05, *** p<0.01.

confirming that the vulnerability-resolution spike is concentrated within the first two weeks and does not reflect a secular trend. Dynamic coefficients confirm no pre-trend and an upward post-discovery shift (Appendix Figure 7).

Table 7: Daily DiD Estimates for Vulnerability-Resolution Keywords per Commit

	±3 Days	±7 Days	±14 Days	±30 Days
Panel A: All Analyzed Repositories				
Vuln.-Res. Kw./Commit	1.36	1.59***	1.05**	0.05
	(0.99)	(0.59)	(0.43)	(0.30)
# Obs	45,858	75,288	128,700	231,182
Panel B: Stacked DiD Repositories				
Vuln.-Res. Kw./Commit	1.74	1.49*	1.47**	0.36
	(1.52)	(0.87)	(0.59)	(0.44)
# Obs	13,787	22,823	39,359	70,560
Repo × Cohort FE	Yes	Yes	Yes	Yes
Relative Day × Cohort FE	Yes	Yes	Yes	Yes

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on the share of commits whose messages contain vulnerability-resolution keywords: *bump*, *upgrade*, *CVE-**, *CVSS*, *vulnerability*, *security-fix*, *security-patch*. Coefficients and standard errors are in percentage points. Panel A uses all repositories with commit histories; Panel B restricts the sample to repositories included in the firm-level stacked DiD analysis. Standard errors in parentheses are clustered at the repository level. * p<0.1, ** p<0.05, *** p<0.01.

While the daily analysis captures the immediate response, vulnerability remediation extends beyond the first few days. Patching dependencies often triggers cascading changes: updating one library may break compatibility with others, requiring additional fixes; security patches may introduce new bugs that must be addressed; and previously deferred maintenance may become necessary when touching affected code. We therefore examine whether the *content* of commits shifts toward remediation-focused work over a longer horizon.

We construct a monthly commit panel by aggregating daily commit data and applying the same stacked difference-in-differences framework used in our firm-level analysis, now over a ±12 month event window to capture the full arc of remediation.[4] As in the daily analysis, we report results for two samples: all repositories with commit histories (464,145 repository-month observations across treated and clean control repositories) and the repositories in the firm-level stacked DiD sample.[5]

For each commit, we create a binary indicator equal to one if its message contains any fix-related keyword—specifically, terms such as *fix*, *bugfix*, *hotfix*, *patch*, *repair*, *resolve*, or *correct* and their

[4]Unlike the code-quality analysis, which uses only the final commit of each month to measure codebase state, the commit-activity analysis aggregates *all* commits within each month to measure development intensity and content.

[5]The commit outcomes are measured directly from repository histories, allowing coverage of repositories without SonarQube code-quality metrics.

inflected forms (e.g., *fixes*, *fixed*, *fixing*, *resolved*, *corrected*). The dependent variable, fix-related keywords per commit, is the monthly proportion of commits for which this indicator equals one, capturing the share of development activity devoted to remediation.

The share of commits containing fix-related language rises by 3.2 percentage points in treated repositories in the broader sample with commit histories ($p < 0.01$; Table 8, column 1). The effect persists in the repositories used in the firm-level stacked DiD analysis, where the share rises by 1.9 percentage points ($p = 0.02$; column 2). Figure 3 shows that the increase emerges in the first month after discovery and remains elevated through the twelve-month event window. The pre-discovery coefficients are close to zero and none is statistically significant. Average commit message length also increases post-discovery (see Appendix Table 17), which could indicate that developers implement more complicated changes that require larger commit messages.

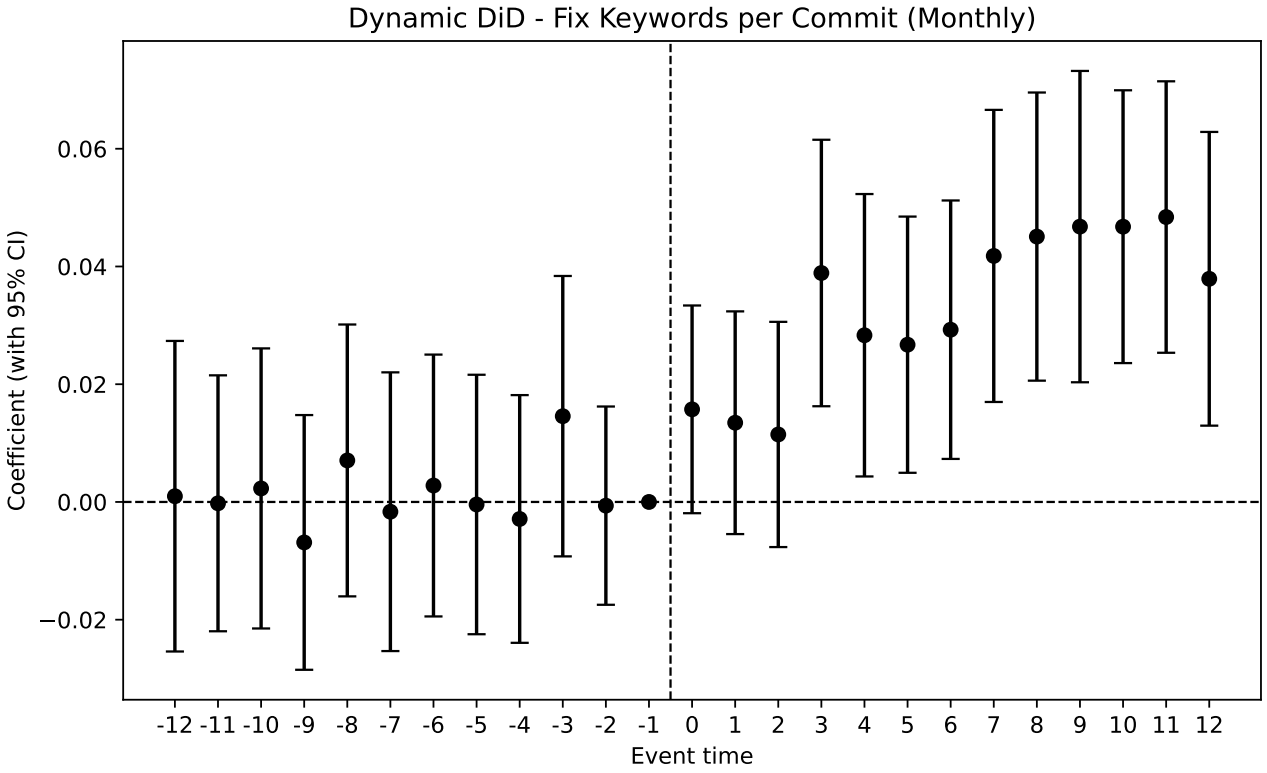


Figure 3: Cybersecurity Vulnerabilities and Fix-Related Commit Activity (±12 Months). Points show monthly stacked DiD estimates for the share of commits containing fix-related language, across all repositories with commit histories. Coefficients are in share units: 0.01 equals one percentage point. Estimates are relative to month −1 before critical vulnerability discovery and include repository-by-cohort and event-month-by-cohort fixed effects. Error bars show 95% confidence intervals using repository-clustered standard errors.

Table 8: Monthly Difference-in-Differences Estimates for Fix-Related Keywords per Commit

	All Analyzed Repositories	Stacked DiD Repositories
	(1)	(2)
Treated × Post	3.16***	1.88**
	(0.62)	(0.82)
Repo × Cohort FE	Yes	Yes
Relative Month × Cohort FE	Yes	Yes
R^2	0.49	0.62
# Obs	464,145	116,365

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on fix-related keywords per commit over a ±12 month event window. The dependent variable is the proportion of commits containing fix-related language (e.g., "fix," "patch," "hotfix"). Coefficients and standard errors are in percentage points. Column (1) uses all repositories with commit histories; column (2) restricts to repositories in the firm-level stacked DiD sample. Standard errors in parentheses are clustered at the repository level. * p<0.1, ** p<0.05, *** p<0.01.

These commit-level results confirm that vulnerability discoveries trigger sustained remediation efforts that persist well beyond the initial security-specific response.

3.3.2 Crowding Out of Innovation

The sustained remediation documented above has direct consequences for the composition of development activity. If engineering resources are persistently absorbed in remediation, we expect a corresponding decline in innovation-oriented work. We test this prediction by examining whether the share of non-remediation commits falls and whether feature-related development stagnates.

We define non-remediation commits as total commits minus those whose messages contain fix, security, or urgent language and those made by automated accounts, such as Dependabot, which proposes dependency updates. The non-remediation share is the percentage of total commits in a repository-month classified as non-remediation.

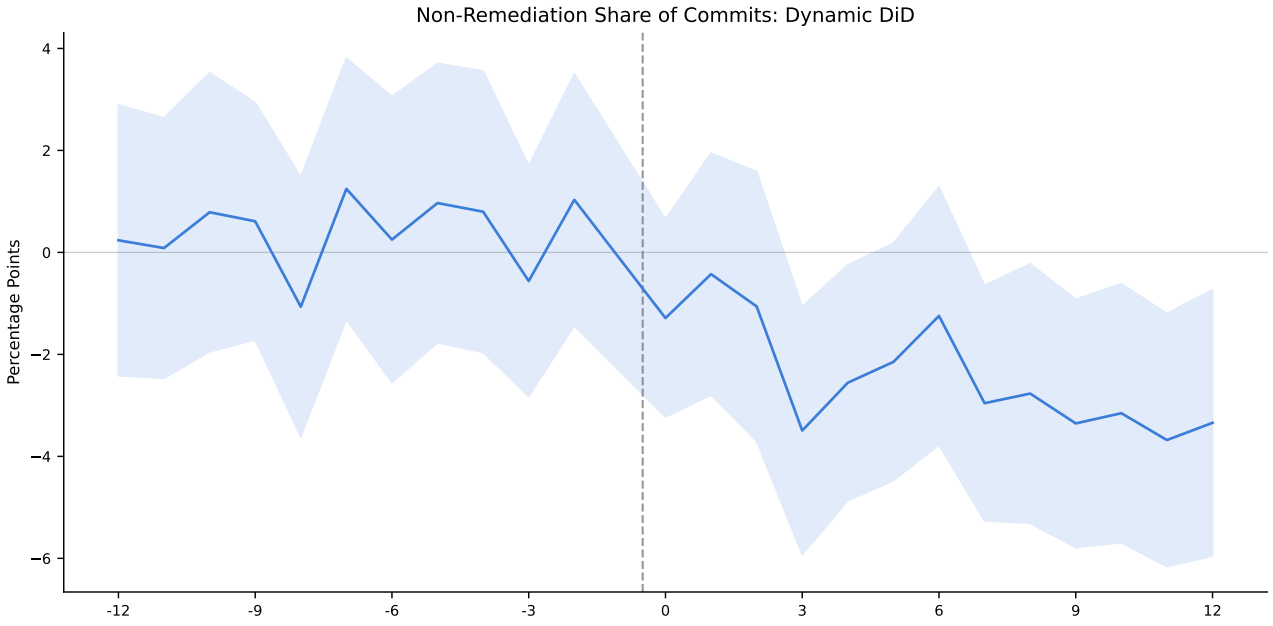


Figure 4: Non-Remediation Share of Commits: Dynamic DiD (±12 Months). The blue line shows event-study estimates; the shaded band shows 95% confidence intervals. The outcome is the non-remediation share of total commits in a repository-month, measured in percentage points, across all repositories with commit histories. Estimates are relative to month −1 before critical vulnerability discovery and include repository-by-cohort and event-month-by-cohort fixed effects. Intervals use repository-clustered standard errors.

Figure 4 previews the headline result: the non-remediation share of commits declines persistently in treated repositories following vulnerability discovery, consistent with engineering resources being diverted from building to maintaining. We examine this composition shift in detail using keyword-based classification for fix, refactoring, and feature categories. Fix-related language rises persistently (confirming Table 8), while refactoring keywords show positive coefficients with individually significant estimates at longer horizons—consistent with deliberate architectural remediation undertaken later (Appendix Figure 8).

Table 9 quantifies the shift. The non-remediation share falls by 2.8 percentage points across all repositories with commit histories ($p < 0.01$) and by 1.7 percentage points in the firm-level stacked DiD sample ($p = 0.08$). The log number of non-remediation commits shows no statistically significant change in either sample ($p = 0.18$ and $p = 0.98$, respectively). The composition of recorded development therefore shifts toward maintenance.

We next examine whether this remediation absorption crowds out innovation-oriented development. We estimate a monthly event-study specification for feature-related commit keywords ("feature," "implement," "enhancement," "introduce," "functionality"), measuring the share of commits containing at least one feature keyword as a percentage of total commits in each repository-month.

Feature-related commit shares decline at longer horizons in the dynamic specification (Appendix Figure 9). The static monthly average is not statistically significant ($\hat{\beta} = -0.15$ p.p., $p = 0.39$).

Column 3 reports the log number of feature-related commits, with estimates of −0.04 in Panel A and −0.11 in Panel B; neither is statistically significant ($p = 0.39$ and $p = 0.18$, respectively). Together with the non-remediation decline, the dynamic pattern supports the crowding-out interpretation.

Table 9: Development Composition Shift: Innovation and Non-Remediation Measures

Dep. Var.:	(1) **Feature/ Commit (‰)**	(2) **Non-Rem./ Commit (‰)**	(3) **ln(Feature-Related Commits)**
Panel A: All Analyzed Repositories			
Treated × Post	-0.15	-2.78***	-0.04
	(0.18)	(0.68)	(0.05)
# Obs	464,131	464,131	203,341
Panel B: Stacked DiD Repositories			
Treated × Post	-0.23	-1.72*	-0.11
	(0.30)	(1.00)	(0.08)
# Obs	117,122	117,122	72,631
Repo × Cohort FE	Yes	Yes	Yes
Rel. Month × Cohort FE	Yes	Yes	Yes

Note: This table presents stacked difference-in-differences estimates for innovation and composition measures over a ±12 month event window. Column 1: feature keywords per commit as a share of total commits in a repository-month (percentage points). Column 2: non-remediation commits as a share of total commits, where non-remediation commits are total commits minus those whose messages contain fix, security, or urgent language and those made by automated accounts such as Dependabot (percentage points). Column 3: natural log of the strictly positive count of commits containing feature-related keywords per repository-month; zero counts are excluded from this column. Panel A uses all repositories with commit histories; Panel B restricts to the firm-level stacked DiD sample. Standard errors in parentheses are clustered at the repository level. * p<0.1, ** p<0.05, *** p<0.01.

Taken together, the evidence connects the immediate remediation response to a persistent shift in development priorities. The decline in non-remediation share and the later decline in feature-related shares show how maintenance demands can crowd out product development, providing an operational link to slower firm growth and fewer acquisition opportunities.

3.3.3 Consequences for Code Quality

Alongside this reallocation of engineering effort, the emergency response also degrades the codebase itself. When teams prioritize speed of remediation over architectural quality, they introduce code that is harder to maintain and extend—precisely the conditions that the software engineering literature associates with the accumulation of technical debt and code complexity (Sculley et al., 2014). We document two manifestations: accumulated technical debt and rising cognitive complexity.

We estimate the same specification as Equation (1) with ln(Technical Debt in USD) as the dependent variable, measuring firm-level TD around our ±12 month event window following the calibration described in Section 2.6.

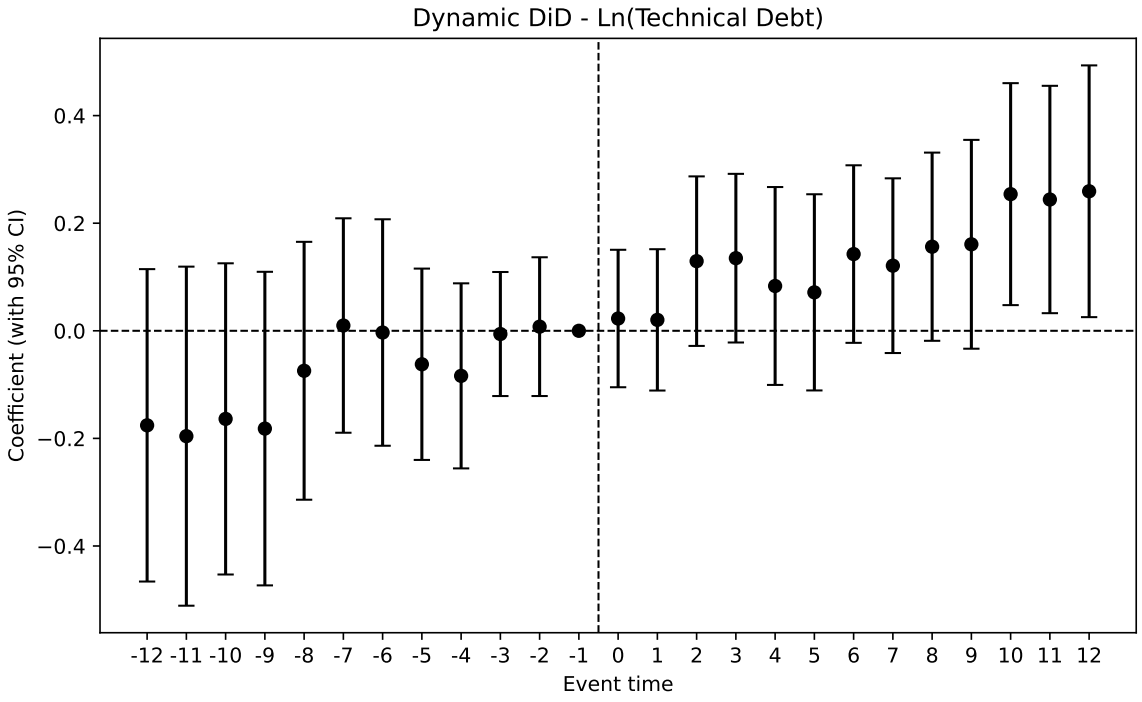


Figure 5: Cybersecurity Vulnerabilities and Technical Debt (Event Time in Months). Points show monthly stacked DiD estimates for log technical debt in dollars in the firm-level estimation sample. Coefficients are in log points and are relative to month −1 before critical vulnerability discovery. The specification includes firm-by-cohort and event-month-by-cohort fixed effects. Error bars show 95% confidence intervals using firm-clustered standard errors.

Table 10 shows that technical debt accumulates approximately 20 percentage points faster among exposed firms in the baseline specification (coefficient 0.20; $p = 0.03$). Adding lines of code gives a coefficient of 0.18 ($p = 0.03$); the language-by-month specification gives 0.17, which is not statistically significant ($p = 0.12$), and the firm-type-by-month specification gives 0.16 ($p = 0.07$). Figure 5 presents the dynamic estimates. This pattern connects vulnerability discoveries to a persistent increase in software maintenance liabilities.

Table 10: Difference-in-Differences Estimates for Ln(Technical Debt in USD)

Dep Var: **Ln(Technical Debt in USD)**	(1)	(2)	(3)	(4)
Treated × Post	0.20**	0.18**	0.17	0.16*
	(0.10)	(0.09)	(0.11)	(0.09)
# Lines of Code		0.00***	0.00***	0.00***
		(0.00)	(0.00)	(0.00)
Firm × Cohort FE	Yes	Yes	Yes	Yes
Relative Month × Cohort FE	Yes	Yes	Yes	Yes
Language × Month FE			Yes	
Firm Type × Month FE				Yes
R^2	0.94	0.94	0.96	0.95
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on technical debt over a ±12 month event window. The dependent variable is the natural log of technical debt in USD, measured by SonarQube static analysis and calibrated to remediation cost benchmarks. The sample and treatment assignment follow Table 4. Column (1) is the baseline specification with firm-by-cohort and event-time-by-cohort fixed effects. Column (2) adds lines of code as a time-varying control for project scale. Column (3) further absorbs language-by-month fixed effects. Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

Cognitive complexity grows 21–24 percentage points faster in treated firms relative to controls in the post-vulnerability period (23–28% higher levels across specifications; largest $p = 0.03$; Table 11), consistent with time-sensitive remediation. We estimate the stacked difference-in-differences specification using ln(Cognitive Complexity) of the codebase as the dependent variable. Cognitive complexity captures the mental effort required to comprehend control flow, nested conditions, and non-linear logic (see Section 2.6 and Appendix A.4 for measurement details). Higher cognitive complexity indicates code that is harder to modify, debug, and extend.

Table 11: Difference-in-Differences Estimates for Ln(Cognitive Complexity)

Dep Var: **Ln(Cognitive Complexity)**	(1)	(2)	(3)	(4)
Treated × Post	0.24***	0.23***	0.22**	0.21**
	(0.09)	(0.08)	(0.10)	(0.08)
# Lines of Code		0.00***	0.00***	0.00***
		(0.00)	(0.00)	(0.00)
Firm × Cohort FE	Yes	Yes	Yes	Yes
Relative Month × Cohort FE	Yes	Yes	Yes	Yes
Language × Month FE			Yes	
Firm Type × Month FE				Yes
R^2	0.94	0.94	0.96	0.95
# Obs	109,790	109,790	109,790	109,790

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on cognitive complexity over a ±12 month event window. The dependent variable is the natural log of cognitive complexity, winsorized at the 1st and 99th percentiles, a SonarQube metric that quantifies the mental effort required to comprehend control flow and nested logic (see Appendix A.4). The sample and treatment assignment follow Table 4. Column (1) is the baseline specification with firm-by-cohort and event-time-by-cohort fixed effects. Column (2) adds lines of code as a time-varying control for project scale. Column (3) further absorbs language-by-month fixed effects. Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects. Nonpositive complexity observations are excluded, and winsorization uses the positive-complexity sample. Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

In panel regressions with firm, month, and language fixed effects, a 10% increase in cognitive complexity is associated with approximately 1.8 percentage points lower subsequent headcount growth ($p = 0.01$; Appendix Table 20, column 4). This relationship is consistent with hastily written emergency patches imposing persistent constraints on organizational growth.

3.4 Robustness

Headcount estimates remain negative and technical-debt estimates remain positive when we omit individual vulnerability cohorts or vary the event window. We assess inference using within-cohort treatment permutations and alternative clustering structures, and report joint Wald tests of pre-treatment coefficients for the firm-level outcomes.

Joint tests of the pre-treatment coefficients are not statistically significant for technical debt ($p = 0.84$), cognitive complexity ($p = 0.90$), acquisition probability ($p = 0.19$) or funding ($p = 0.18$). For headcount, technical debt, and cognitive complexity, the test uses all eleven monthly pre-

treatment interaction coefficients; acquisition and funding use three quarterly coefficients, matching their respective event-study specifications. For headcount, the joint test gives $W(11) = 17.4$ ($p = 0.09$). The pre-period point estimates for headcount are slightly positive.

To assess whether any single vulnerability event drives our results, we conduct a leave-one-cohort-out analysis, re-estimating the baseline specification 38 times, each time dropping one cohort from the stacked sample. For headcount, the coefficient ranges from -0.09 to -0.07, tightly bracketing the full-sample estimate of -0.08 (Figure 6). For technical debt, the range is $[0.17, 0.22]$ relative to the full-sample coefficient of 0.20 (Appendix Figure 10). No single cohort is pivotal for either outcome.

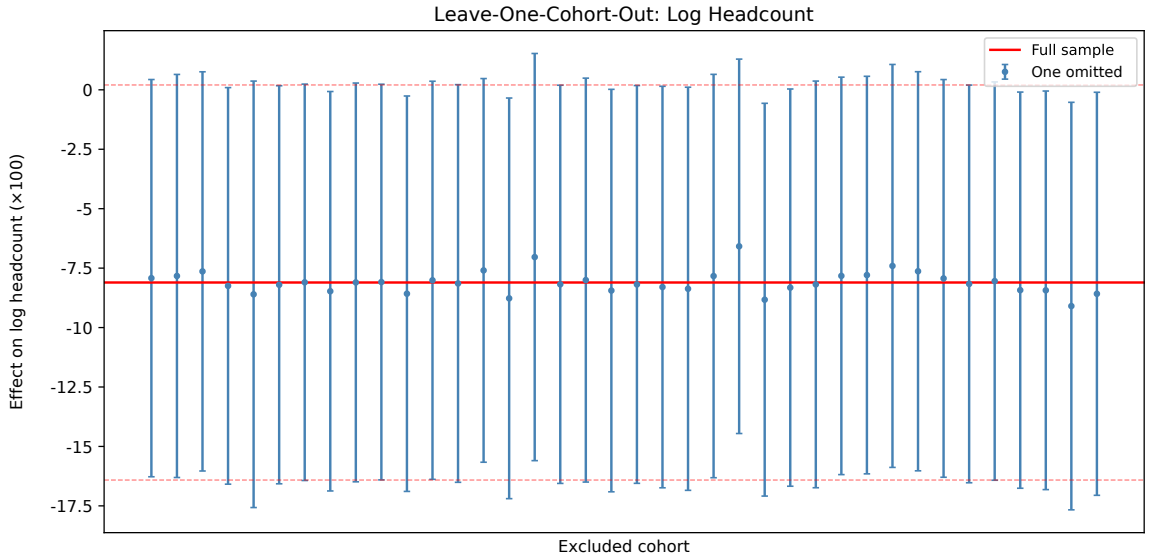


Figure 6: Leave-One-Cohort-Out Estimates: Ln(Headcount). Each blue point is the baseline stacked DiD coefficient after omitting one vulnerability cohort, using the firm-level estimation sample. The vertical axis reports 100 times the log-headcount coefficient. Blue error bars show 95% confidence intervals using firm-clustered standard errors. The solid red line is the full-sample estimate; dashed red lines mark its 95% confidence interval.

We assess the estimates with randomization inference, permuting treatment labels within each cohort 1,000 times while preserving the treated count. The permutation p-values are 0.07 for headcount (Appendix Figure 11) and below 0.01 for technical debt (Appendix Figure 12).

Across symmetric event windows from six to eighteen months, headcount estimates remain negative and technical-debt estimates positive (Appendix Table 24). Using firm-clustered standard errors throughout, the headcount estimates have $p = 0.01$ in the six- and nine-month windows and $p = 0.05$ in the twelve-month window, while the eighteen-month estimate is not statistically significant ($p = 0.23$). The technical-debt estimates have $p = 0.04$, 0.03 and 0.03 in the nine-, twelve- and eighteen-month windows and $p = 0.06$ in the six-month window. Appendix A.6 also reports clustering and upgrade-type comparisons.

4 Conclusion

Public discoveries of critical vulnerabilities in third-party software dependencies impose costs that extend beyond the immediate security task. Across 610 open-core software firms, our codebase histories connect these discoveries to slower headcount growth and fewer acquisition opportunities. Commit-level evidence traces the engineering response: activity rises immediately after discovery, fix-related work remains elevated for twelve months, and the share of non-remediation development falls by 2.8 percentage points. Technical debt accumulates approximately 20 percentage points faster in the baseline specification, while cognitive complexity grows 21–24 percentage points faster across specifications. Together, these findings connect an external security shock to persistent demands on engineering resources and the maintenance of software assets.

The implications extend through the software supply chain. The firms we study produce databases, security libraries and developer platforms used by other organizations; open-source components appear in 96% of audited commercial codebases (Synopsys, 2024), and the demand-side value of widely used open-source software is estimated at $8.8 trillion (Hoffmann, Nagle, and Zhou, 2024). Applying a dollar-denominated measure of technical debt to vulnerability discoveries brings cybersecurity into questions of firm growth, software valuation and acquisition due diligence. Vulnerability discoveries can shift engineering priorities from building new capabilities toward maintaining existing ones, with lasting consequences for firms that rely on shared software.

References

Amir, E., S. Levi, and G. Livne. 2018. Do firms underreport information on cyber-attacks? evidence from capital markets. *Review of Accounting Studies* 23:1177–206.

Baker, A. C., D. F. Larcker, and C. C. Wang. 2022. How much should we trust staggered difference-in-differences estimates? *Journal of Financial Economics* 144:370–95.

Callaway, B., and P. H. Sant'Anna. 2021. Difference-in-differences with multiple time periods. *Journal of Econometrics* 225:200–30. Themed Issue: Treatment Effect 1.

Campbell, G. A. 2023. Cognitive complexity: A new way of measuring understandability. Working Paper, SonarSource. Version 1.7.

Conti, A., V. Gupta, J. Guzman, and M. P. Roche. 2023. Better keep the twenty dollars: Incentivizing innovation in open source. Working Paper 24-014, Harvard Business School. Revised January 2025. Also available as NBER Working Paper Series, No. 31668, September 2023.

Conti, A., C. Peukert, and M. Roche. 2025. Beefing it up for your investor? engagement with open source communities, innovation, and startup funding: Evidence from github. *Organization Science* 36:1551–73.

Crosignani, M., M. Macchiavelli, and A. Silva. 2023. Pirates without borders: The propagation of cyberattacks through firms' supply chains. *Journal of Financial Economics* 147:432–48.

Cunningham, W. 1992. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA'92)*, 29–30. ACM. doi:10.1145/157709.157715.

Dimas, J., and L. Pnevmatikos. 2026. Venture capital, engineering quality, and technical debt. *SSRN Electronic Journal* .

Eisfeldt, A. L., and D. Papanikolaou. 2013. Organization capital and the cross-section of expected returns. *The Journal of Finance* 68:1365–406.

Ewens, M., R. H. Peters, and S. Wang. 2025. Measuring intangible capital with market prices. *Management Science* 71:407–27.

Fershtman, C., and N. Gandal. 2011. Direct and indirect knowledge spillovers: the "social network" of open-source projects. *The RAND Journal of Economics* 42:70–91.

Florackis, C., C. Louca, R. Michaely, and M. Weber. 2023. Cybersecurity risk. *The Review of Financial Studies* 36:351–407.

Gormley, T. A., and D. A. Matsa. 2011. Growing out of trouble? corporate responses to liability risk. *The Review of Financial Studies* 24:2781–821.

———. 2016. Playing it safe? managerial preferences, risk, and agency conflicts. *Journal of Financial Economics* 122:431–55.

Hoffmann, M., F. Nagle, and Y. Zhou. 2024. The value of open source software. Strategy Unit Working Paper 24-038, Harvard Business School.

Imbens, G. W., and D. B. Rubin. 2015. *Causal inference for statistics, social, and biomedical sciences: An introduction*. Cambridge University Press. doi:10.1017/CBO9781139025751.

Krasner, H. 2022. Cost of poor software quality in the U.S. : A 2022 report. Working Paper, Consortium for Information & Software Quality (CISQ). Accessed: March 25, 2025.

Kruchten, P., R. L. Nord, and I. Ozkaya. 2012. Technical debt: From metaphor to theory and practice. *IEEE Software* 29:18–21.

Letouzey, J.-L. 2012. The SQALE method for evaluating technical debt. In *2012 Third International Workshop on Managing Technical Debt (MTD)*, 31–6. Zurich, Switzerland. doi:10.1109/MTD.2012.6225997.

Li, Z., P. Avgeriou, and P. Liang. 2015. A systematic mapping study on technical debt and its management. *Journal of Systems and Software* 101:193–220.

McCabe, T. J. 1976. A complexity measure. *IEEE Transactions on Software Engineering* SE-2:308–20.

McKinsey & Company. 2023. Breaking technical debt's vicious cycle to modernize your business. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/breaking-technical-debts-vicious-cycle-to-modernize-your-business. Accessed: March 25, 2025.

Nagle, F. 2019. Open source software and firm productivity. *Management Science* 65:1191–215.

Sculley, D., G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, and M. Young. 2014. Machine learning: The high interest credit card of technical debt. In *SE4ML: Software Engineering for Machine Learning (NIPS 2014 Workshop)*.

Stripe, I. 2018. The developer coefficient. https://stripe.com/newsroom/stories/developer-coefficient. Accessed: March 25, 2025.

Sun, L., and S. Abraham. 2021. Estimating dynamic treatment effects in event studies with heterogeneous treatment effects. *Journal of Econometrics* 225:175–99. Themed Issue: Treatment Effect 1.

Synopsys. 2024. 2024 open source security and risk analysis report. Working Paper, Synopsys, Inc. Based on audit of 1,067 commercial codebases by Black Duck Audit Services.

U.S. Census Bureau. 2025. 2022 SUSB annual data tables by establishment industry. https://www.census.gov/data/tables/2022/econ/susb/2022-susb-annual.html. Statistics of U.S. Businesses, NAICS 511210 (Software Publishers). Accessed: October 26, 2026.

A Appendix

All logged variables use the natural logarithm of strictly positive values. Zero and nonpositive observations are excluded from specifications using the corresponding logged variable.

Table 12: Variable Definitions

Variable	**Definition**
A. Firm outcomes and characteristics	
Headcount	Total number of employees at firm i in period t, constructed from LinkedIn position histories. Our main growth outcome uses ln(Headcount).
Software engineer (SWE) headcount	Number of employees in software engineering roles at firm i in period t, based on a job-title mapping applied to LinkedIn position histories.
Funding round	Indicator equal to 1 if firm i receives a financing event in period t (Seed Round, Early Stage VC, Late Stage VC, PE Growth/Expansion, Accelerator/Incubator, or Angel investment), based on PitchBook deal records.
Acquisition	Indicator equal to 1 in and after the first observed Merger/Acquisition or Buyout/LBO event for firm i, based on PitchBook deal records.
Is Alive	Indicator equal to 1 until the first period in which PitchBook lists firm i as out of business; 0 thereafter.
Founding year	Year the organization was founded (PitchBook).
Product type	Firm product-category classification (10 categories; see Table 18) constructed from firm descriptions using Gemini and GPT. We retain their common classification when they agree, use Gemini when they disagree, and use the available classification when only one model classifies the firm. These are grouped into five broader firm types for Column (4) fixed effects.
$Growth_{t+24}$	Headcount growth between a firm's current monthly observation and its 24th subsequent monthly observation: $\text{Headcount}_{i,t+24}/\text{Headcount}_{i,t} - 1$.

Continued on next page

Table 12 – continued from previous page

Variable	Definition
B. Cybersecurity shocks and stacked DiD variables	
Critical dependency vulnerability (event)	Public discovery of a critical-severity vulnerability (CVSS score ≥ 9) affecting a third-party software dependency used by a firm, sourced from the GitHub Advisory Database (GitHub-reviewed subset; merged with NVD metadata). The event month is the discovery month.
Common Vulnerability Scoring System (CVSS)	Standardized 0–10 severity scale used to quantify the severity and exploitability of software vulnerabilities. We define critical discoveries as those with CVSS ≥ 9.
Treated	Indicator equal to 1 if firm i is exposed to the cohort's critical dependency vulnerability; treated firms are first exposed at $t = 0$ within each stacked cohort.
Post	Indicator equal to 1 for periods strictly after the cohort event month ($t \geq 1$) and 0 otherwise; the discovery month ($t = 0$) is coded as pre-period since treatment exposure is partial during that month.
Treated $\times$ Post ($did_{i,c,t}$)	Main treatment indicator in the stacked difference-in-differences specification, equal to $\text{Treated}_{i,c} \times \text{Post}_{c,t}$.
Cohort (c)	Event-specific cohort defined by a unique (dependency, discovery month) pair; cohorts are stacked to form the analysis sample.
Relative time (t)	Event time measured relative to each cohort's discovery month (reported at either monthly or quarterly frequency depending on the specification).
Cohort $\times$ firm fixed effects ($\chi_{c,i}$)	Firm-by-cohort fixed effects that absorb time-invariant firm characteristics within each stacked cohort.
Time $\times$ cohort fixed effects ($\delta_{c,t}$)	Time-by-cohort fixed effects that absorb common shocks affecting all firms within a cohort at event time t.
Upgrade severity (Major/Minor)	Classification of the dependency update as Major or Minor, based on its recorded version components; used to stratify the upgrade comparisons.

Continued on next page

Table 12 – continued from previous page

Variable	Definition
C. Codebase scale and engineering-quality metrics (SonarQube)	
Lines of code (LoC)	Total number of non-comment lines of source code in the firm's product repository(ies) at time t (SonarQube NCLOC). When aggregating to the firm level, we sum across all repositories observed for the firm in period t. Logged versions use ln(LoC).
Open issues	Stock of unresolved SonarQube issues (reliability, security, or maintainability) currently active in the codebase at time t. Logged versions use ln(Open Issues).
Critical violations	Stock of critical-severity code violations (reliability, security, or maintainability) flagged by SonarQube at time t. Logged versions use ln(count).
Bugs per kLoC	Number of SonarQube-detected bug issues per thousand non-comment lines of code at time t.
Duplicated lines	Total number of duplicated lines of code across the firm's repository(ies) at time t, as reported by SonarQube.
Dominant programming language	The most common programming language across the firm's repositories in period t, used to construct language-by-cohort and language-by-month fixed effects.
D. Security and technical debt metrics (SonarQube)	
Technical debt (TD, USD)	Calibrated debt stock $\alpha \times$ SQALE Minutes, where SQALE (Software Quality Assessment based on Lifecycle Expectations) is SonarQube's remediation index, measured in minutes. We use a common multiplier $\alpha \approx 33.04$ dollars per SQALE minute (Appendix A.3). Logged versions use ln(TD). When aggregating to the firm level, we sum TD across all repositories observed for the firm in period t.
Technical Debt per Line of Code	Technical debt divided by lines of code (LoC): TD/LoC.

Continued on next page

Table 12 – continued from previous page

Variable	Definition
Technical Debt Velocity	Technical debt on new code per new line of code, constructed as new SQALE minutes times the same dollar calibration factor, divided by new lines when new lines > 0.
Cognitive Complexity	A SonarQube metric that quantifies how difficult code is to understand and maintain. Unlike cyclomatic complexity (which counts decision paths), cognitive complexity weights structures based on mental effort required: nested conditions, non-linear control flow, and recursive patterns receive higher scores. When aggregating to the firm level, we sum cognitive complexity across all repositories. Logged versions use ln(Cognitive Complexity).
Security Hotspots	Number of security-sensitive code segments flagged by SonarQube as requiring manual review to assess vulnerability. We sum the number of hotspots across a firm's repositories in period t to obtain the firm's total hotspot stock.
New security issues (High)	Number of new high-severity security issues detected by SonarQube in the firm's repository(ies) during period t, under SonarQube's alternative (Multi-Quality Rule) classification; these are not a subset of the traditional vulnerability count. Logged versions use ln(count).
Vulnerabilities per kLoC	Number of SonarQube-detected vulnerability issues per thousand non-comment lines of code at time t.
Security issues (High) per kLoC	Number of high-severity security issues per thousand non-comment lines of code at time t, under SonarQube's alternative (Multi-Quality Rule) classification; not a subset of vulnerabilities per kLoC.
E. Commit-level activity metrics	
Commits	Total number of commits to the repository in period t. Logged versions use ln(Commits).
Fix keywords per commit	Ratio of commits containing fix/patch-related keywords (e.g., fix, patch, hotfix, bugfix, resolve, repair, correct) to total commits in period t. Captures remediation-related development activity.

Continued on next page

Table 12 – continued from previous page

Variable	**Definition**
Average message length	Mean length (in characters) of commit messages in period t. Logged versions use ln(Avg Message Length). Longer messages may indicate more detailed documentation of changes.
Days with commits	Number of days in period t with at least one commit. Controls for development intensity when analyzing per-commit metrics.
Feature keywords per commit	Proportion of commits containing feature-related keywords (e.g., feature, implement, enhancement) in period t. Captures innovation-oriented development activity.

Note: Codebase metrics are measured using SonarQube static analysis.

A.1 CVSS Scoring Framework

The Common Vulnerability Scoring System (CVSS) is an open, industry-standard framework for communicating the characteristics and severity of software vulnerabilities. CVSS is owned and managed by FIRST.Org, Inc. (FIRST), a US-based non-profit organization whose mission is to help computer security incident response teams across the world. The system provides a standardized, vendor-agnostic methodology for quantifying vulnerability severity, enabling consistent assessment across different software products, vendors, and organizational environments.

We primarily use CVSS v3.1 Base scores in this study; when v3.1 is unavailable, we use the most recent available version (v3.0 or, rarely, v2.0). CVSS produces numerical scores ranging from 0 to 10, where higher scores indicate greater severity. These scores are derived from three metric groups: Base, Temporal, and Environmental. The Base metrics capture the intrinsic characteristics of a vulnerability that remain constant over time and across user environments, assuming a reasonable worst case impact across different deployed environments. The Temporal metrics reflect characteristics that change over time, such as the availability of exploit code or official fixes and workarounds. The Environmental metrics allow organizations to customize scores based on the importance of affected assets and the presence of mitigating controls in their specific computing environment.

The CVSS specification explicitly assumes that scoring occurs after the attacker has already located and identified the vulnerability; therefore, the difficulty of vulnerability discovery is not factored into the severity score.

Base Metrics and Scoring. The Base Score is the foundation of CVSS and reflects the severity of a vulnerability according to its intrinsic characteristics. It is composed of two sub-groups: Exploitability metrics and Impact metrics.

The *Exploitability metrics* capture the ease and technical means by which a vulnerability can be exploited:

- **Attack Vector (AV)**: The context by which exploitation is possible (Network, Adjacent, Local, or Physical). Holding other metrics constant, Network attack vector yields the highest contribution to the Base score.
- **Attack Complexity (AC)**: The conditions beyond the attacker's control that must exist to exploit the vulnerability (Low or High). All else equal, Low complexity vulnerabilities contribute more to the Base score.
- **Privileges Required (PR)**: The level of privileges an attacker must possess before exploiting the vulnerability (None, Low, or High). All else equal, vulnerabilities requiring no privileges contribute most to the Base score.

- **User Interaction (UI)**: Whether exploitation requires action from a user other than the attacker (None or Required). All else equal, vulnerabilities requiring no user interaction contribute more to the Base score.

The *Impact metrics* capture the direct consequences of successful exploitation on three dimensions:

- **Confidentiality (C)**: The impact on the confidentiality of information resources (High, Low, or None). High indicates complete loss of confidentiality.
- **Integrity (I)**: The impact on the integrity and trustworthiness of information (High, Low, or None). High indicates the attacker can modify any/all protected files.
- **Availability (A)**: The impact on the availability of the affected component (High, Low, or None). High indicates complete denial of access to resources.

An additional metric, **Scope (S)**, captures whether a vulnerability in one component can affect resources beyond its security scope (Unchanged or Changed). A scope change occurs when the impact breaches a security boundary, affecting components managed by different security authorities.

The Base Score formula combines these metrics through a weighted calculation that produces a score from 0.0 to 10.0. Vulnerabilities with easier exploitation paths (e.g., network-accessible, low complexity, no privileges required) and higher impacts (e.g., complete loss of confidentiality, integrity, and availability) receive higher scores.

Qualitative Severity Ratings. CVSS maps numerical scores to qualitative severity ratings to facilitate communication and prioritization. Table 13 presents the standard mapping used throughout this study.

Table 13: CVSS v3.1 Qualitative Severity Rating Scale

Rating	CVSS Score Range
None	0.0
Low	0.1 – 3.9
Medium	4.0 – 6.9
High	7.0 – 8.9
Critical	9.0 – 10.0

Critical-Severity Vulnerabilities. We study vulnerabilities with CVSS Base Scores of 9.0 or above, the critical-severity threshold under CVSS v3.x. When only CVSS v2.0 scores are available, we apply the same numeric cutoff as a high-severity screen. Restricting the range of severity improves

comparability across shocks. Critical vulnerabilities are commonly prioritized for remediation, making their disclosure more likely to prompt rapid triage and repairs that can divert engineering resources from other activities.

Many critical-severity vulnerabilities are characterized by high exploitability (such as network-based attack vectors with low complexity) combined with severe potential impacts on confidentiality, integrity, or availability. For example, a vulnerability allowing remote code execution without authentication that results in complete system compromise would typically receive a critical rating. However, critical scores can arise from different combinations of exploitability and impact metrics, particularly when scope changes occur. In our sample of third-party dependency vulnerabilities (see Table 14 in the Appendix), the mean CVSS Base score is 9.4, with all treated vulnerabilities scoring between 9.0 and 10.0 by construction.

Vector Strings. CVSS communicates vulnerability characteristics through vector strings—compact textual representations of all metric values. A vector string begins with "CVSS:3.1/" followed by abbreviated metric names and values separated by forward slashes. For example:

`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

This vector represents a vulnerability with Network attack vector, Low complexity, no Privileges Required, no User Interaction, Unchanged scope, and High impact on Confidentiality, Integrity, and Availability. This combination produces a Base Score of 9.8, placing it firmly in the critical-severity range. The vector string provides a transparent, reproducible representation of how a given severity score was derived, allowing readers to understand which characteristics drive the overall assessment.

Application in This Study. We obtain CVSS scores and associated metadata from two widely used sources: the GitHub Advisory Database and the National Vulnerability Database (NVD), operated by the U.S. National Institute of Standards and Technology (NIST). We rely on CVSS Base scores as reported in these databases. We prioritize version first (v3.1 > v3.0 > v2.0); within a given version, if multiple sources report scores, we use the GitHub Advisory Database score, as it reflects the GitHub security review process specific to the open-source ecosystem our firms operate in. Given that the vast majority of vulnerabilities in our sample period (2016–2025) have v3.0 or v3.1 scores, the number of v2-only scored vulnerabilities is negligible. Our focus on Base Scores (rather than Temporal or Environmental scores) reflects the fact that Base metrics are publicly reported and consistent across all affected organizations, enabling a standardized measurement of shock severity. By restricting our analysis to critical-severity vulnerabilities affecting third-party dependencies, we study disclosure events with high standardized severity that are plausibly exogenous to individual firm characteristics.

Table 14: Vulnerability Severity Distribution (GitHub Advisory Database)

Severity	Count	Frequency (%)
MODERATE	10,485	44.12
HIGH	8,317	35.00
CRITICAL	3,416	14.37
LOW	1,547	6.51

Note: Counts refer to the GitHub-reviewed subset of the GitHub Advisory Database across all ecosystems. Our causal shock sample restricts to CRITICAL severity (CVSS $\geq$ 9.0) and matched firm exposures.

Table 15: Vulnerability Distribution by Software Ecosystem (GitHub Advisory Database)

Ecosystem	Count	Frequency (%)	Cumulative Frequency (%)
Maven	5,764	24.25	24.25
Packagist	4,788	20.15	44.40
npm	4,117	17.32	61.73
PyPI	3,883	16.34	78.06
Go	2,501	10.52	88.59
crates.io	1,018	4.28	92.87
RubyGems	893	3.76	96.63
NuGet	681	2.87	99.50
SwiftURL	37	0.16	99.65
GitHub Actions	36	0.15	99.80
Hex	36	0.15	99.95
Pub	11	0.05	100.00

Notes: Counts refer to the GitHub-reviewed subset of the GitHub Advisory Database. Each ecosystem corresponds to a language-specific package management system: *Maven* (Java ecosystem), *Packagist* (PHP), *npm* (JavaScript/TypeScript), *PyPI* (Python), *Go* (Go modules), *crates.io* (Rust), *RubyGems* (Ruby), *NuGet* (.NET including C# and F#), *SwiftURL* (Swift Package Manager), *GitHub Actions* (reusable automated testing and deployment workflows), *Hex* (Elixir and Erlang), and *Pub* (Dart/Flutter).

Table 16: Vulnerability Discoveries by Year (GitHub Advisory Database)

Year	Count	Frequency (%)
2017	221	0.93
2018	785	3.30
2019	745	3.13
2020	1,398	5.88
2021	2,610	10.98
2022	8,523	35.86
2023	3,332	14.02
2024	3,654	15.38
2025	2,497	10.51

Note: Counts refer to the GitHub-reviewed subset of the GitHub Advisory Database across all severity levels.

A.2 Commit Message Length Analysis

Table 17: Difference-in-Differences Estimates for Commit Message Length

	Ln(Avg Message Length)	
	(1)	(2)
Treated × Post	0.07***	0.07***
	(0.03)	(0.03)
Days with Commits		0.00***
		(0.00)
Repo × Cohort FE	Yes	Yes
Relative Month × Cohort FE	Yes	Yes
R^2	0.76	0.76
# Obs	326,733	326,733

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on average commit message length. The dependent variable is the natural log of average commit message length (in characters) per repository-month. The sample consists of repository-month observations for treated repositories and clean control repositories. Standard errors in parentheses are clustered at the repository level. * p<0.1, ** p<0.05, *** p<0.01.

A.3 Technical Debt Calibration

Let $S_{i,t}$ denote SQALE minutes and $L_{i,t}$ non-comment lines of code. With development time $\ell = 30$ minutes per line, wage $w = \$55$ per hour and target $\varphi = 0.20$, the dollar multiplier is

$$\alpha = \varphi \frac{w}{60} \ell \frac{\text{Median}(L)}{\text{Median}(S)}, \qquad \text{TD}_{i,t} = \alpha S_{i,t}.$$

The medians of lines of code and SQALE minutes are taken separately over the firm-month SonarQube panel, which gives $\alpha \approx 33.04$ dollars per SQALE minute. Because the multiplier is common to all firms and months, it leaves log-change comparisons unchanged.

A.4 Cognitive Complexity: Overview

Cognitive Complexity is a metric developed by SonarSource (and implemented in SonarQube) to measure the understandability of code (Campbell, 2023). Unlike Cyclomatic Complexity—formulated by McCabe (1976) to measure testability by counting linearly independent paths through a program—Cognitive Complexity is designed to quantify the mental effort required for a human to read and comprehend code. This distinction is economically meaningful: two methods with identical Cyclomatic Complexity scores can differ dramatically in how difficult they are to understand, modify, and maintain. The following provides a high-level summary; see Campbell (2023) for the complete specification.

Core Principles. Cognitive Complexity is assessed according to three basic rules:

1. **Ignore shorthand structures.** Features that condense multiple statements into more readable forms (e.g., null-coalescing operators) do not increment complexity. Similarly, method definitions themselves receive no baseline increment, unlike Cyclomatic Complexity. This rewards coding practices that improve readability.

2. **Increment for breaks in linear flow.** Structures that interrupt the natural top-to-bottom, left-to-right reading of code require additional mental effort. These include loops (`for`, `while`, `do-while`), conditionals (`if`, ternary operators), `switch` statements, `catch` blocks (but not `try` or `finally`), jumps to labels (`goto`, `break LABEL`, `continue LABEL`), sequences of binary logical operators, and recursion. Note that `else if` and `else` receive hybrid increments but do not incur nesting penalties, as the mental cost is already paid when reading the initial `if`.

3. **Increment for nesting.** When flow-breaking structures are nested inside one another, additional increments are assessed for each level of nesting. This captures the intuition that deeply nested code imposes compounding cognitive load.

Key Differences from Cyclomatic Complexity. Several design choices distinguish Cognitive Complexity from its predecessor:

- **Switch statements**: Cyclomatic Complexity treats each `case` as a separate branch, assigning one increment per case. Cognitive Complexity assigns a single increment for the entire `switch`, reflecting that comparing a single variable to explicit literal values is cognitively simpler than parsing an `if-else if` chain with arbitrary conditions.

- **Logical operators**: Rather than incrementing for each binary logical operator, Cognitive Complexity increments once per *sequence* of like operators. The expression `a && b && c && d` receives one increment, while `a || b && c || d` receives three (one per sequence of like operators), reflecting that mixed operators require more careful parsing.

- **Nesting penalties**: Cyclomatic Complexity does not account for nesting depth. Cognitive Complexity applies cumulative nesting increments, so a conditional nested inside a loop inside another conditional incurs progressively higher penalties.

- **Modern constructs**: Cognitive Complexity handles `try/catch` blocks (incrementing for `catch` but not `try` or `finally`), lambdas (which increment nesting level but not structural complexity), and recursion (which receives a fundamental increment per method in the recursion cycle).

- **Aggregation**: Because Cognitive Complexity does not assign a baseline increment to each method (unlike Cyclomatic Complexity's minimum score of one), aggregate scores at the class or application level are meaningful. A domain class with many simple getters and setters will have low aggregate complexity, while a class with complex control flow will score high—regardless of method count.

Interpretation. SonarQube reports aggregate Cognitive Complexity at the repository level (summing across all methods in the codebase). In our analysis, we sum this repository-level measure across all repositories a firm commercializes to obtain firm-level Cognitive Complexity, then apply a $\ln(x)$ transformation for regression analysis, excluding nonpositive observations. Higher scores indicate codebases that are more difficult for developers to understand, modify, and extend. When teams face time pressure—such as during vulnerability remediation—they may write code that solves the immediate problem but introduces nested conditionals, complex error handling, or convoluted control flow. This accumulated complexity represents a form of "cognitive debt" that imposes ongoing costs on future development.

A.5 Additional Tables and Figures

Table 18: Firm Types (Based on Core Product)

Firm Type	Frequency (%)	Cumulative (%)
Developer Tools & Platforms	24.2	24.2
Cloud & Infrastructure	14.5	38.7
Databases & Storage	14.1	52.9
Analytics & Observability	13.5	66.4
Security & Identity	10.9	77.3
Enterprise & Business	5.5	82.8
AI & Machine Learning	5.5	88.3
Collaboration & Productivity	4.9	93.2
Other	4.5	97.7
Testing & Quality Assurance	2.3	100.0

Note: Percentages describe the 488 firms with SonarQube measures. Product categories are assigned from firm descriptions using the classification rule in Section 2.

Table 19: Geographic Distribution of Firms

Country	Frequency (%)	Cumulative Frequency (%)
United States	61.63	61.63
Germany	4.16	65.79
United Kingdom	3.54	69.34
France	3.08	72.42
Canada	2.16	74.58
Italy	1.85	76.43
China	1.39	77.81
Netherlands	1.23	79.04
Israel	1.08	80.12
Ireland	0.92	81.05
India	0.77	81.82
Other	4.93	86.75
Missing	13.25	100.00

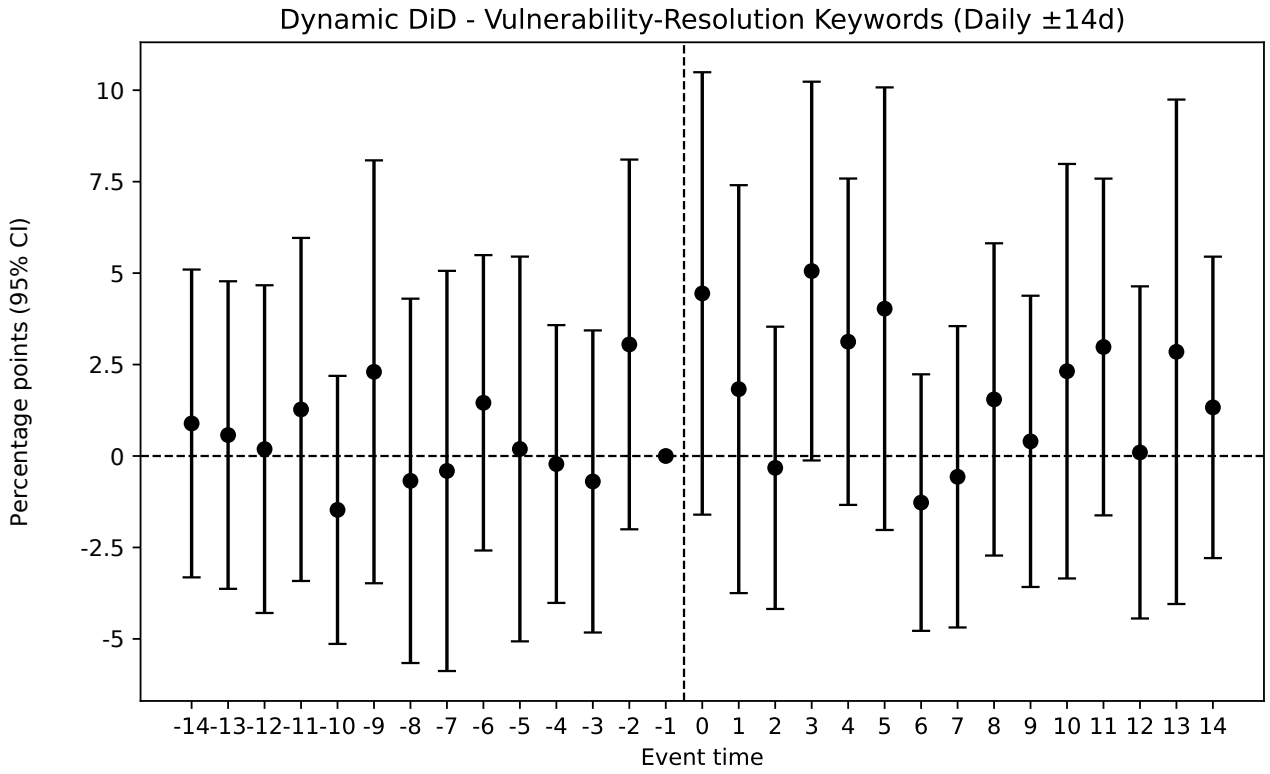


Figure 7: Cybersecurity Vulnerabilities and Vulnerability-Resolution Commit Activity (±14 Days). Points show daily stacked DiD estimates for the share of commits containing vulnerability-resolution language, using repositories in the firm-level stacked DiD sample. Coefficients are in percentage points. Estimates are relative to day −1 before critical vulnerability discovery and include repository-by-cohort and event-day-by-cohort fixed effects. Error bars show 95% confidence intervals using repository-clustered standard errors.

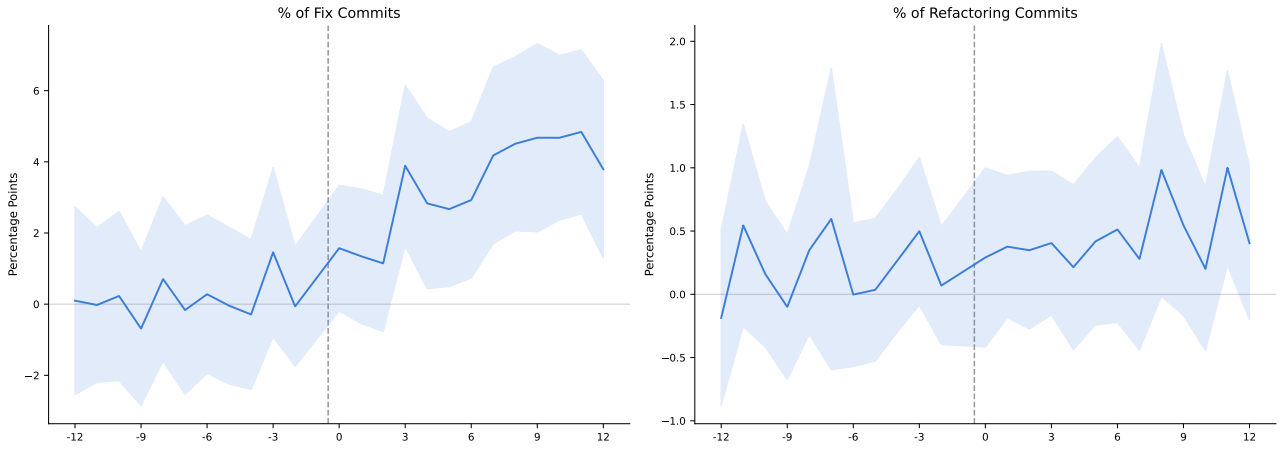


Figure 8: Cybersecurity Vulnerabilities and Development Composition (±12 Months). Panels show fix-related and refactoring commit shares. Shaded bands show 95% confidence intervals. Each outcome is the share of total commits in a repository-month containing the stated language, measured in percentage points, across all repositories with commit histories. Estimates are relative to month −1 before critical vulnerability discovery and include repository-by-cohort and event-month-by-cohort fixed effects. Intervals use repository-clustered standard errors.

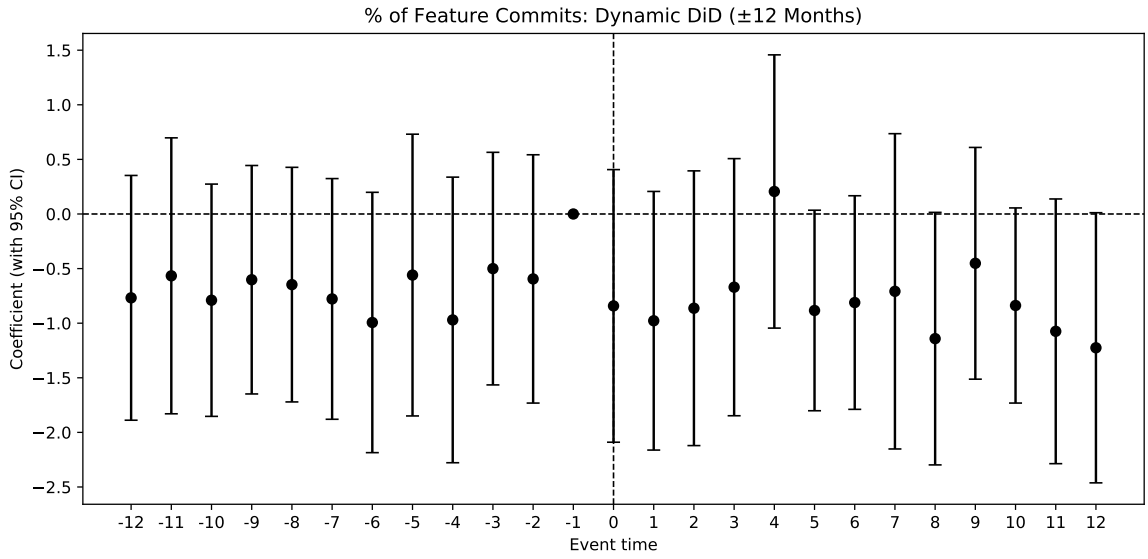


Figure 9: Feature Development: Dynamic DiD (±12 Months). Points show monthly stacked DiD estimates for the share of commits containing feature-related language, across all repositories with commit histories. Coefficients are in percentage points. Estimates are relative to month −1 before critical vulnerability discovery and include repository-by-cohort and event-month-by-cohort fixed effects. Error bars show 95% confidence intervals using repository-clustered standard errors.

Table 20: Cognitive Complexity and Firm Growth

Dep Var: $Growth_{t+24}$	(1)	(2)	(3)	(4)	(5)	(6)
Ln(Cognitive Complexity)	-0.06	-0.17**	-0.17**	-0.19**	-0.32	-0.35
	(0.05)	(0.07)	(0.07)	(0.08)	(0.25)	(0.28)
Ln(New Security Issues - High)					-0.13	-0.12
					(0.11)	(0.11)
Total Security Hotspots						0.00
						(0.00)
Firm FE		Yes	Yes	Yes	Yes	Yes
Month FE			Yes	Yes	Yes	Yes
Language FE				Yes	Yes	Yes
R^2	0.00	0.42	0.42	0.43	0.77	0.77
# Obs	12,484	12,483	12,483	12,482	501	501

Note: This table presents OLS estimates of the relationship between cognitive complexity and subsequent headcount growth. The dependent variable is the proportional change in headcount between the current monthly observation and the 24th subsequent monthly observation, $\text{Headcount}_{i,t+24}/\text{Headcount}_{i,t} - 1$. The sample is the broader firm-level monthly panel (not the stacked DiD sample). Column (1) is a pooled OLS regression with no fixed effects. Columns (2)–(6) progressively add firm, month, and programming language fixed effects, as well as controls for the flow of new high-severity security issues (from SonarQube) and total security hotspots. Logged counts use strictly positive observations; columns (5)–(6) therefore use months with positive high-severity security-issue flows. Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

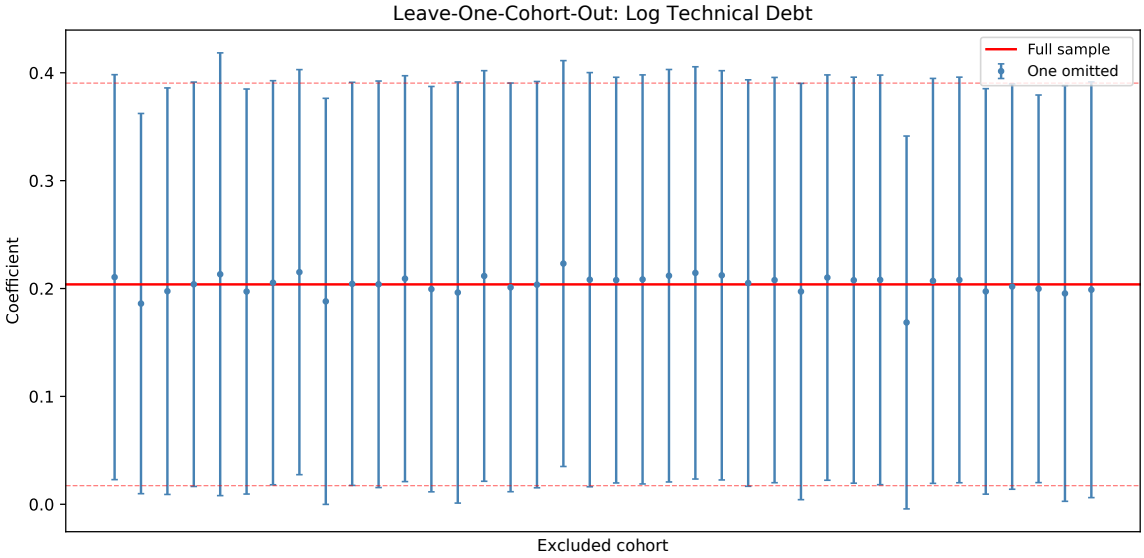


Figure 10: Leave-One-Cohort-Out Estimates: Ln(Technical Debt). Each blue point is the baseline stacked DiD coefficient for log technical debt after omitting one vulnerability cohort, using the firm-level estimation sample. Coefficients are in log points. Blue error bars show 95% confidence intervals using firm-clustered standard errors. The solid red line is the full-sample estimate; dashed red lines mark its 95% confidence interval.

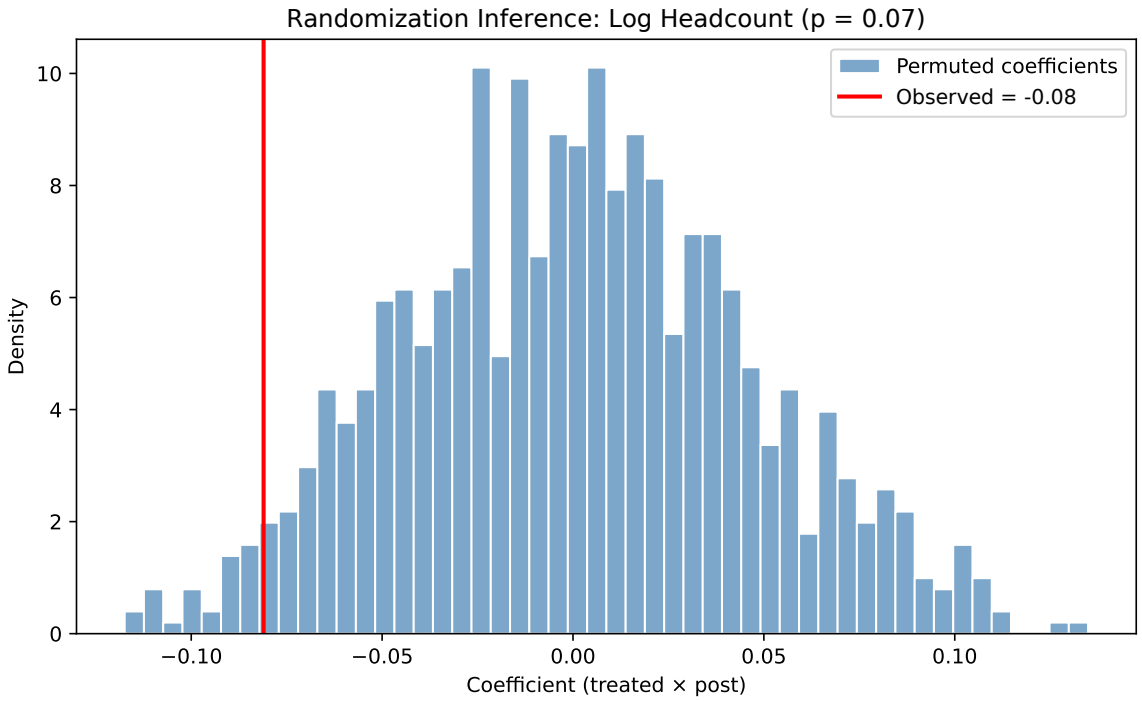


Figure 11: Randomization Inference: Ln(Headcount). The histogram shows baseline stacked DiD coefficients for log headcount from 1,000 permutations of treatment labels within each cohort, preserving the number of treated firms. The sample is the firm-level estimation sample. Coefficients are in log points. The red vertical line marks the estimate under the observed treatment assignment.

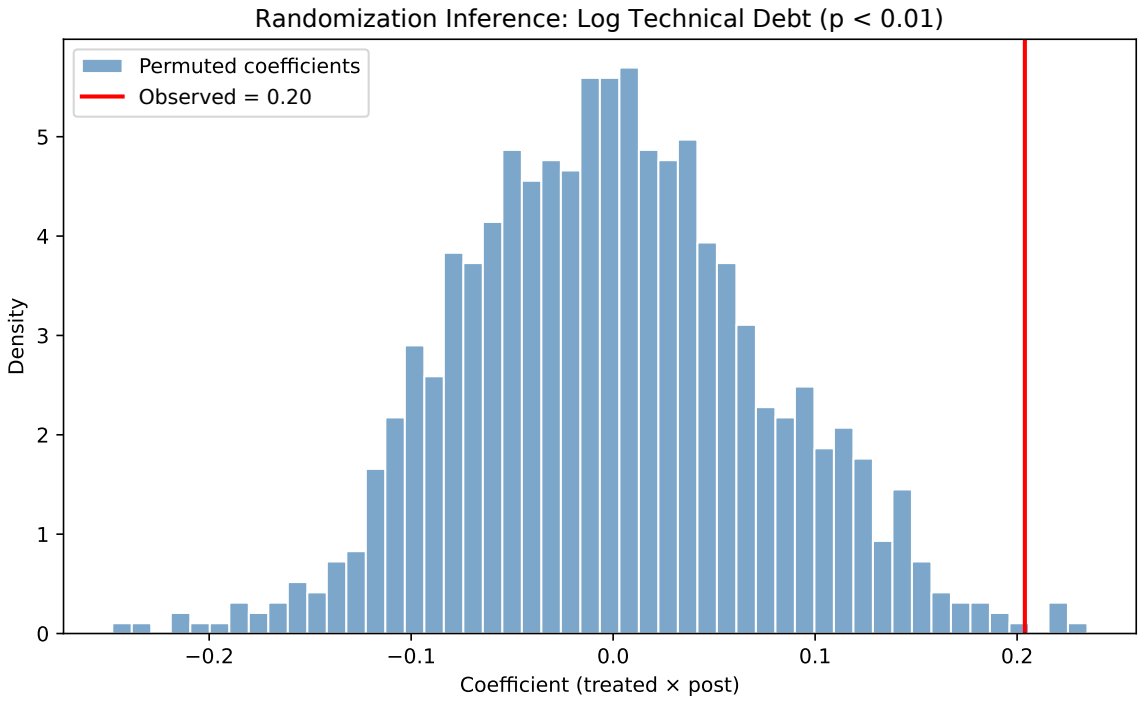


Figure 12: Randomization Inference: Ln(Technical Debt). The histogram shows baseline stacked DiD coefficients for log technical debt from 1,000 permutations of treatment labels within each cohort, preserving the number of treated firms. The sample is the firm-level estimation sample. Coefficients are in log points. The red vertical line marks the estimate under the observed treatment assignment.

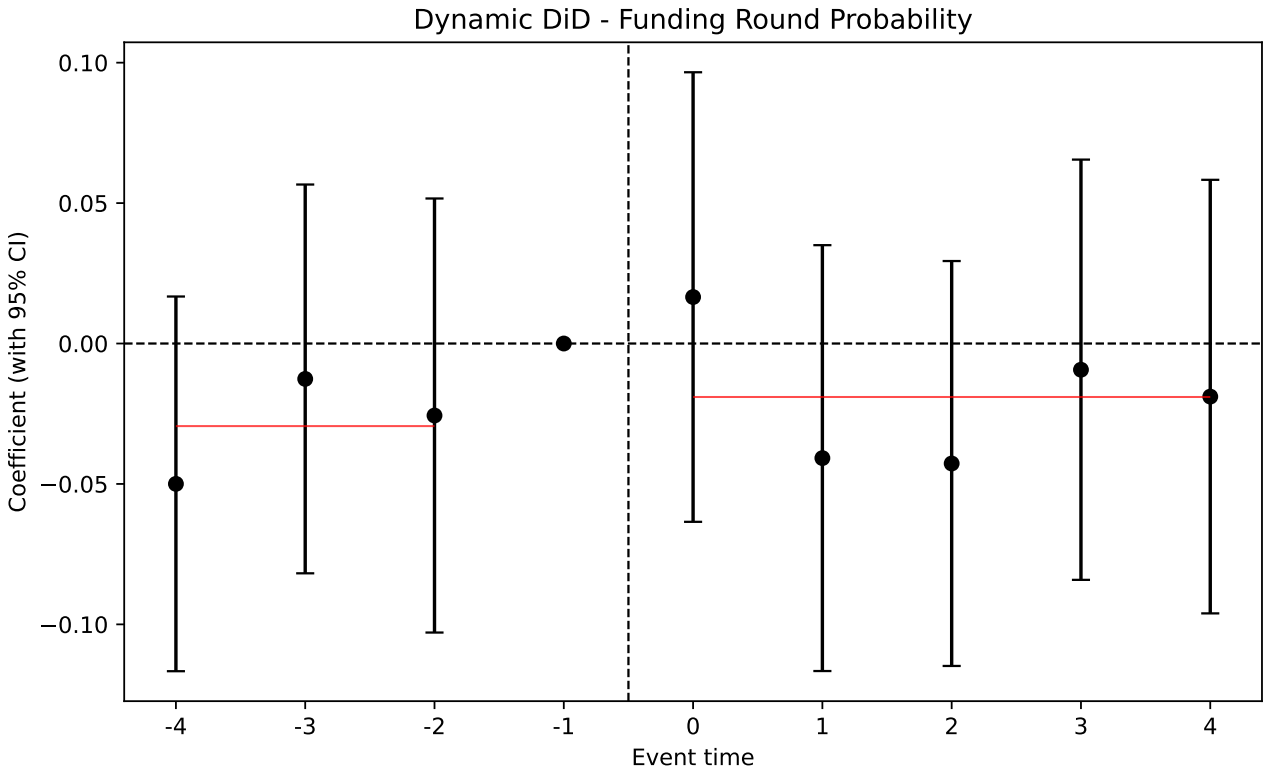


Figure 13: Cybersecurity Vulnerabilities and Funding Rounds (Event Time in Quarters). Points show stacked DiD estimates for whether a firm received any funding round within an event-time quarter, using the firm-level estimation sample. Coefficients are in probability units: 0.01 equals one percentage point. Estimates are relative to months −4 through −2; the bin at quarter 0 spans months −1 through +1. The specification includes firm-by-cohort and event-month-by-cohort fixed effects. Error bars show 95% confidence intervals using firm-clustered standard errors. Red horizontal segments show average coefficients before quarter 0 and from quarter 0 onward.

Table 21: Difference-in-Differences Estimates for Funding

Dep Var: **Funding Round (Pre/Post)**	(1)	(2)	(3)	(4)
Treated × Post	2.84	0.99	1.00	2.94
	(6.31)	(6.38)	(6.38)	(6.22)
# Lines of Code			0.00	0.00
			(0.00)	(0.00)
Firm × Cohort FE	Yes	Yes	Yes	Yes
Relative Month × Cohort FE	Yes	Yes	Yes	Yes
Language × Month FE		Yes	Yes	
Firm Type × Month FE				Yes
R^2	0.67	0.68	0.68	0.67
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on firm funding over a ±12 month event window. The dependent variable equals one if the firm received a funding round (from PitchBook) at any point within the pre- or post-discovery phase, and is repeated across months within that phase. Coefficients and standard errors are in percentage points. The sample and treatment assignment follow Table 4. Column (1) includes firm-by-cohort and event-time-by-cohort fixed effects only. Column (2) adds language-by-month fixed effects. Column (3) further controls for project scale (lines of code). Column (4) replaces language fixed effects with firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

Table 22: Difference-in-Differences Estimates for Survival

Dep Var: **Is Alive**	(1)	(2)	(3)	(4)
Treated × Post	0.23	0.21	0.21	0.22
	(0.15)	(0.14)	(0.14)	(0.16)
Ln(Open Issues)		0.06	0.03	0.01
		(0.04)	(0.03)	(0.04)
Ln(Lines of Code)			0.04	0.08
			(0.04)	(0.07)
Firm × Cohort FE	Yes	Yes	Yes	Yes
Relative Month × Cohort FE	Yes	Yes	Yes	Yes
Firm Type × Month FE				Yes
R^2	0.95	0.95	0.95	0.95
# Obs	114,393	114,393	114,393	114,393

Note: This table presents stacked difference-in-differences estimates of the effect of critical vulnerability discoveries on firm survival over a ±12 month event window. The dependent variable is an indicator equal to one if the firm is alive (not closed or defunct per PitchBook) in a given month. Coefficients and standard errors are in percentage points. The sample and treatment assignment follow Table 4. Column (1) is the baseline specification with firm-by-cohort and event-time-by-cohort fixed effects. Column (2) adds the natural log of open issues as a time-varying control. Column (3) further controls for project scale (natural log of lines of code). Column (4) adds firm-type-by-calendar-month fixed effects. Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

A.6 Additional Robustness Tests

This appendix reports additional robustness tests supplementing Section 3.4. We present pre-trend diagnostics, window sensitivity, clustering sensitivity, and dose-response heterogeneity. Leave-one-cohort-out analysis and randomization inference are reported in the main text.

Pre-Trend Joint Tests. Table 23 reports joint Wald tests of the null hypothesis that all pre-treatment dynamic coefficients are jointly equal to zero. For headcount, technical debt, and cognitive complexity, the test uses all eleven monthly pre-treatment interaction coefficients; acquisition and funding use three quarterly coefficients, matching their respective event-study specifications. The tests are not statistically significant for technical debt ($p = 0.84$), cognitive complexity ($p = 0.90$), acquisition probability ($p = 0.19$) or funding ($p = 0.18$). For headcount, the joint test gives $W(11) = 17.4$ ($p = 0.09$). As discussed in Section 3.4, the pre-period point estimates for headcount are slightly positive.

Table 23: Joint Wald Tests of Pre-Treatment Dynamic Coefficients

Outcome	Pre-Period Coefficients	Test Statistic	p-Value
Ln(Headcount)	11	17.37	0.09
Ln(Technical Debt)	11	6.37	0.84
Ln(Cognitive Complexity)	11	5.41	0.90
Acquisition	3	4.70	0.19
Funding	3	4.82	0.18

Note: This table reports joint Wald tests of the null hypothesis that all pre-treatment dynamic coefficients are jointly equal to zero. For headcount, technical debt, and cognitive complexity, the test uses all eleven monthly pre-treatment interaction coefficients (months −12 through −2, with month −1 as the reference). For acquisition and funding, the test uses three quarterly pre-treatment coefficients, matching those outcomes' event-study specifications. The tests use firm-clustered covariance, as in the main specifications. Wald statistics use a $\chi^2(q)$ reference distribution, where q is the number of pre-period coefficients. The complexity test uses log complexity winsorized at the first and ninety-ninth percentiles within the estimation sample.

Window Sensitivity. Table 24 compares the baseline specification across symmetric six-, nine-, twelve- and eighteen-month windows, using firm-clustered standard errors throughout. Headcount estimates are negative and technical-debt estimates positive across all four windows; the table reports their magnitudes, precision and observation support.

Table 24: Sensitivity of Main Results to Event Window Length

Window	Outcome	Coefficient	SE	p-Value	Obs
±6 months	Ln(Headcount)	−0.07**	(0.03)	0.01	70,943
±6 months	Ln(Technical Debt)	0.13*	(0.07)	0.06	70,943
±9 months	Ln(Headcount)	−0.09**	(0.04)	0.01	97,046
±9 months	Ln(Technical Debt)	0.16**	(0.08)	0.04	97,046
±12 months	Ln(Headcount)	−0.08*	(0.04)	0.05	114,393
±12 months	Ln(Technical Debt)	0.20**	(0.10)	0.03	114,393
±18 months	Ln(Headcount)	−0.07	(0.05)	0.23	143,933
±18 months	Ln(Technical Debt)	0.23**	(0.11)	0.03	143,933

Note: This table examines the sensitivity of the main firm-level results to the choice of event window. Each row re-estimates the Column (1) baseline stacked DiD specification (firm-by-cohort and event-time-by-cohort fixed effects, no additional controls) with a different symmetric window around the vulnerability discovery date. The main results use ±12 months. Each window uses the control assignments in its existing estimation panel. The six-, nine-, twelve- and eighteen-month panels contain 40, 40, 38 and 36 cohorts, respectively. Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

Clustering Sensitivity. Our main specifications cluster standard errors at the firm level. Table 25 reports the baseline specifications under three alternative clustering structures: firm-only (our main approach), two-way clustering at the firm and cohort level, and cohort-only clustering. The point estimate is identical across structures (only standard errors change). Across the three structures, the largest p-value, under firm-level clustering, is 0.05 for headcount and 0.03 for technical debt. Firm-level clustering is the most conservative of the three approaches, producing the largest standard errors.

Table 25: Sensitivity of Main Results to Clustering Structure

Outcome	Clustering	Coefficient	SE	p-Value
Ln(Headcount)	Firm only	−0.08*	(0.04)	0.05
Ln(Headcount)	Cohort + Firm	−0.08**	(0.03)	0.02
Ln(Headcount)	Cohort only	−0.08***	(0.03)	<0.01
Ln(Technical Debt)	Firm only	0.20**	(0.10)	0.03
Ln(Technical Debt)	Cohort + Firm	0.20***	(0.07)	<0.01
Ln(Technical Debt)	Cohort only	0.20***	(0.06)	<0.01

Note: This table examines the sensitivity of the main firm-level results to the choice of standard error clustering. Each row reports the Column (1) baseline stacked DiD specification (±12 month window, firm-by-cohort and event-time-by-cohort fixed effects) with a different clustering structure. The point estimate is mechanically identical across structures; only standard errors change. The main results use firm-level clustering, which accounts for serial correlation within firm and cross-cohort correlation from control firms appearing in multiple cohorts. * p<0.1, ** p<0.05, *** p<0.01.

Dose-Response Heterogeneity. Table 26 compares firms requiring Major and Minor dependency updates. Headcount estimates are −0.09 for Major and −0.07 for Minor updates, and neither is statistically significant ($p = 0.14$ and $p = 0.15$, respectively). Technical-debt estimates are 0.18 for Major updates ($p = 0.08$) and 0.22 for Minor updates, which is not statistically significant ($p = 0.11$). The pooled interaction test for technical debt is not statistically significant ($p = 0.84$).

Table 26: Dose-Response Heterogeneity: Upgrade Severity

Upgrade Type	Outcome	Coefficient	SE	p-Value
Major	Ln(Headcount)	−0.09	(0.06)	0.14
Minor	Ln(Headcount)	−0.07	(0.05)	0.15
Major	Ln(Technical Debt)	0.18*	(0.11)	0.08
Minor	Ln(Technical Debt)	0.22	(0.14)	0.11

Note: This table examines whether treatment effects vary with the complexity of the required remediation. Treated firms are stratified by Major/Minor upgrade labels. The control group is the full set of untreated firms; only the treated subsample varies. All specifications use the Column (1) baseline stacked DiD (±12 month window, firm-by-cohort and event-time-by-cohort fixed effects). Standard errors in parentheses are clustered at the firm level. * p<0.1, ** p<0.05, *** p<0.01.

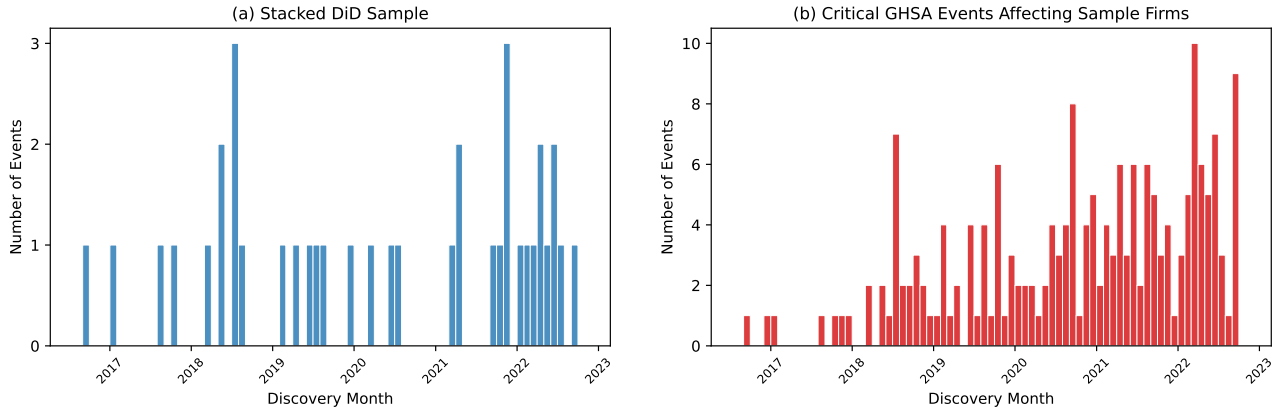


Figure 14: Distribution of Critical Vulnerability Events Over Time. Panel (a) shows the 38 cohort events used in our stacked DiD sample. Panel (b) shows the 189 critical-severity events (CVSS $\geq$ 9) recorded in the GitHub Advisory Database that affect packages used in our firms' repositories over the same period. Both panels share the same time axis (September 2016 to September 2022). The temporal distribution of our DiD cohorts broadly mirrors the rising trend in critical vulnerability discoveries, suggesting that our sample of shocks is representative of the critical disclosures affecting our firms rather than concentrated in an unusual period.

Table 27: Sample Attrition

Restriction	**Firms**	**Firms Dropped**
OSSRank universe	648	—
Matched repository	636	12
Valid SonarQube metrics	488	148
Single repository, headcount > 1	298	190
Stacked cohort panel	295	3
DiD estimation sample	284	11

Note: This table documents sample construction. *OSSRank universe* includes all 648 open-core companies identified via OSSRank rankings. *Matched repository* includes the 636 firms for which we matched at least one primary GitHub or GitLab repository; for the remaining 12 firms, no commercialized project was matched to a repository. *Valid SonarQube metrics* restricts to firms with at least one month of positive lines of code, excluding repositories whose programming languages or build configurations are not supported by our static analysis pipeline. *Single repository, headcount* > 1 retains firms that commercialize exactly one open-source repository, so that codebase metrics unambiguously capture the firm's core product, and that have peak headcount exceeding one employee, excluding individual-developer side projects. *Stacked cohort panel* applies the treatment definition: treatment is the firm's first critical-severity dependency vulnerability classified as requiring a major or minor upgrade; a control firm's first such discovery must occur at least 12 months after the cohort event, or the firm must have no such discovery. This yields 295 firms before applying repository-month measurement requirements. *DiD estimation sample* additionally requires positive lines of code in the matched repository-month and finite, positive headcount. The final estimation sample contains 284 firms (117 treated and 167 controls) across 38 vulnerability cohorts.